# Acquisition, Reconstruction, and World Models: A Compute-First Architecture for Physical AI Data Generation

Chien-Ping Lu

May 4, 2026

**Abstract**

Physical AI needs enormous amounts of data, but the field is caught between two imperfect stories. Classical simulation promises physical correctness but is expensive, graphics-hardware-bound, and difficult to scale across AI accelerator fleets. Generative world models promise cheap interactive futures, but their predictions are only physical inside a validated envelope. This paper argues for a third architecture: *physical acquisition plus explicit state plus scalable reconstruction and prediction*. Calibrated simulators, sensors, and low-sample path tracers provide sparse physical evidence; persistent 3D/4D state stores geometry, radiance, motion, visibility, variance, and confidence; classical filters, Gaussian splatting, and neural/world-model components then denoise, supersample, interpolate, predict, and propose counterfactuals. The system returns to a physical or sensor oracle when uncertainty, rare events, or task risk exceed the learned model's validation envelope. Crucially, synthetic data generation and model training form a closed loop: the current robot model acts in the simulated or reconstructed world, its failures and uncertainty drive new acquisitions, and the resulting synthetic data revises the next model. This framing explains several otherwise separate trends. DLSS-class rendering already treats rasterization and ray tracing as sparse acquisition followed by tensor-side reconstruction [1]–[4]. However, DLSS is compute-heavy in implementation but proprietary and graphics-stack-packaged in deployment. vkGSplat therefore names its open 3DGS counterpart **Deep Generation Super Sampling (DGSS)**: a Vulkan-seed-buffer-backed Gaussian/radiance reconstruction layer that consumes sparse seeds and serves denoising, temporal reuse, super-resolution, frame interpolation, generative completion, and confidence-gated oracle refresh. Robotics SDG is throughput-bound and latency-tolerant, so it can trade fixed-function graphics latency for compute portability. More importantly, SDG has an amortized world-building phase that games do not: it can spend seconds, minutes, or hours establishing static geometry, materials, lighting priors, BVHs, Gaussian maps, and high-bounce radiance caches before running many simulation and acquisition episodes. World-model systems such as Genie and Cosmos are compatible with, rather than replacements for, calibrated physics and sensors when they act as priors, rollout generators, or proposal mechanisms [5]–[8]. We use **vkGSplat** as the reference implementation of this philosophy. The name is intentional: **vk** means the system embraces standard Vulkan as the application, workload, shader, resource, synchronization, and tooling leverage point, and **GSplat** means 3D Gaussian Splatting becomes DGSS, the open reconstruction layer that replaces the architectural role of proprietary DLSS for SDG. This is not a proposal for a new graphics API, a semantic extension of Vulkan, or a replacement standard. The architecture is acquisition-backend neutral: a conventional Vulkan renderer on graphics hardware is a valid and often best acquisition oracle. vkGSplat instead asks how much of the work behind the existing Vulkan contract can be lowered to CUDA-class software kernels and persistent Gaussian state when data generation, reconstruction, training, and world-model inference should remain on tensor-rich accelerators. The internal semantic goal is **Vulkan-compatible world semantics**: geometry, primitive identity, material and lighting state, sensor pose, motion, uncertainty, and Gaussian/radiance history reconstructed behind a standard Vulkan boundary. It is materialized as a device-resident persistent state cache, an optimization and reconstruction substrate behind Vulkan, not a new API surface or a near-term standard. The CUDA software path is useful when the renderer must expose custom sample statistics and explicit Gaussian state updates, but those are implementation artifacts behind the standard contract, not new application semantics. vkGSplat is not the whole thesis; it is the concrete test of whether physical-evidence acquisition, explicit Gaussian state, PyTorch-centered orchestration, and tensor-rich hardware can form a practical data-generation substrate. As a falsifiable milestone rather than a reported result, we target $\geq 100$ **images/s/H100** at $1024\times1024$ from a captured 3DGS scene, $\leq 0.5$ **J/image** on H100-class hardware, and **within** $2\times$ **of an RTX-class graphics-hardware baseline (L40) on dollars per image**, while preserving downstream sim-to-real task accuracy.

**Index Terms**

Physical AI, synthetic data generation, world models, acquisition and reconstruction, Deep Generation Super Sampling, DLSS-class reconstruction, Vulkan, software rendering, 3D Gaussian splatting, robotics, AI accelerators.

## I. Introduction

Physical AI data generation is often framed as a choice between two extremes. One extreme is calibrated simulation: physics engines, path tracers, sensor models, and digital twins that try to measure the virtual world with known error bars. The other extreme is learned world modeling: generative systems that predict plausible futures, transform scenes, and create interactive rollouts without explicitly replaying every physical process. Both are necessary, and neither is sufficient alone. Physics without reconstruction is too expensive to scale; generation without physical acquisition can drift away from the regimes where robotics needs trust.

This paper takes a different position. The right unit of design is not a renderer, a simulator, or a world model by itself. It is an *acquisition–state–reconstruction loop*. A calibrated simulator, physical sensor, rasterizer, or low-sample path tracer acquires

sparse evidence about the world. An explicit state representation stores that evidence across space, time, views, modalities, uncertainty, and task context. Reconstruction and world-model modules then denoise, supersample, interpolate, fill missing views, predict futures, and propose counterfactuals. When the state is under-supported — low sample count, shallow path-depth evidence, high variance, disocclusion, distribution shift, or safety-critical task risk — the system returns to a physical or sensor oracle. Put simply: AI should reduce the cost of acquiring, storing, reconstructing, and querying physical truth; it should not silently replace physical truth.

This is not an open-loop dataset factory. The robot or embodied agent being trained is part of the data-generation loop. A current perception, policy, or world model acts in the simulated or reconstructed world; its rollouts expose uncertain states, failures, near misses, and rare contacts; vkGSplat then acquires or reconstructs the evidence needed to turn those events into new training examples. The revised model changes the next rollout distribution. Synthetic data generation and model training are therefore coupled, not staged as "render once, train later."

This loop also changes the time structure of rendering. A game renderer must rebuild enough of the world to satisfy the next display frame. A SDG system can first build a world model: static geometry, materials, light-transport priors, BVHs, 3DGS state, and sensor calibration can be computed with generous budgets before any policy rollout or synthetic camera trajectory begins. The online simulation loop then updates dynamic state — robot motion, movable objects, contact, deformables, light changes, material edits, and occlusions — against that amortized static base. This is a qualitative difference, not merely a slower frame budget. It means vkGSplat should optimize the whole lifecycle, not just per-frame rendering.

Modern graphics already points in this direction. In DLSS-class real-time pipelines, rasterization and ray tracing increasingly produce only a low-resolution, temporally jittered, spatially sparse seed. Tensor-side reconstruction performs super-resolution, anti-aliasing, denoising, and frame generation [1]–[3]. Ray Reconstruction makes the same point for path-traced lighting: sparse physical rays acquire evidence, then a neural model reconstructs the missing samples [4]. The important lesson for Physical AI is not that a denoiser is magically physical. It is that low-sample acquisition can remain grounded in the correct transport process while reconstruction amortizes the cost. Lowering samples per pixel preserves the target integral in expectation; lowering bounce depth changes the physical model and must be tracked as bias.

This distinction is central to vkGSplat. DLSS is fundamentally a compute reconstruction pipeline — it runs on tensor hardware and consumes motion, depth, exposure, and history — but it is packaged as a proprietary graphics feature inside the RTX/game-rendering stack. That packaging is rational for games, yet it is the wrong boundary for AIDC-native synthetic data generation. vkGSplat keeps the useful architectural lesson and changes the substrate: integrated 3DGS/radiance state replaces the role of DLSS as the reconstruction layer. We call this layer *Deep Generation Super Sampling* (DGSS): deep because learned resolvers may be attached, generation because the target is SDG and world-model feedback rather than only display upscaling, and super sampling because sparse physical evidence is reconstructed across space, time, views, and Gaussian/radiance history. DGSS remains open, scene-aware, persistent across views, and directly connected to CUDA/PyTorch training workflows.

World-model research fits the same loop when its role is stated carefully. Classical world models learn compact predictive dynamics for control [5]; Genie-style and Cosmos-style systems scale toward interactive or generative environments for embodied agents [6]–[8]. These models can replace many expensive simulator calls inside a validation envelope: rollouts, variations, priors, proposals, and counterfactuals. They should not replace the calibrated physics, sensor, or rendering oracle when the output is a measurement whose errors matter. A useful world model proposes; the acquisition oracle checks, refreshes, or corrects when confidence is weak.

The hardware implication is just as important. A generic architecture should allow visual acquisition to come from the best available source: a physical camera, an offline path tracer, a conventional Vulkan renderer running on graphics hardware, or a software renderer running on AI compute. Today's leading SDG stacks — NVIDIA Omniverse/Isaac Sim/Replicator and Cosmos [8], Unreal-based platforms, Blender-based pipelines [9], [10] — usually use the hardware Vulkan or RTX path, and that is the right baseline when those GPUs are available and co-location is not a concern. The open question is whether the same acquisition contract can also execute efficiently on tensor-rich AI accelerators (NVIDIA H100/B200, Google TPU, AWS Trainium, and related chips), which dominate FLOPS, memory bandwidth, model deployment, and procurement. If acquisition is a sparse seed and reconstruction is the scalable value center, then a compute-first implementation becomes plausible. The question is not whether graphics hardware disappears, but when the Vulkan execution contract should be allowed to move off fixed-function graphics units.

There is also a blunt economic motivation. The AI data center (AIDC) has become a hundred-billion-dollar capital program: Microsoft described an approximately $80B FY2025 investment in AI-enabled datacenters, Alphabet guided to roughly $75B of 2025 capital expenditures, and Meta raised its 2025 capital-expenditure outlook to $70–72B while warning that 2026 growth would be driven primarily by infrastructure costs [11]–[13]. Industry analysts describe the same pressure at the data-center level: Omdia's Data Center World 2026 material frames the period as a "trillion dollar" AIDC boom, estimates $6.07T of total IT spending in 2026, reports that 72.9% of surveyed organizations expect IT-budget growth in 2026, and projects data-center investment approaching $1.9T by 2030 under a 20% CAGR scenario [14]. The same deck points to rack-scale AI systems, 800VDC power distribution, BESS, self-generation, and liquid cooling as near-term data-center design pressures. We use this as market and infrastructure evidence, not as a law of nature. The implication is that SDG architecture must fit a capital program that is already large, power-constrained, and operations-heavy. In that world, the goal is not to beat NVIDIA's RTX approach on

its own terms. RTX/Omniverse/Isaac-style SDG is the natural baseline when a graphics fleet is already the production substrate. The problem is that many organizations are already locked into a massive AIDC buildout, and disrupting that buildout to add a parallel graphics island is economically and operationally expensive. The economic bet behind vkGSplat is therefore simpler: make synthetic data generation a normal AI data-center workload, scheduled, containerized, monitored, billed, and optimized on the same accelerator fleet that already runs training and inference.

This also reframes the comparison with NVIDIA's ecosystem. NVIDIA has a powerful heterogeneous answer: Omniverse and OVX systems for physically realistic simulation, graphics, and industrial digital twins, and DGX/DGX Cloud for AI-factory training and inference [15], [16]. That split is technically coherent, and it is the correct reference architecture for sites that can justify a dedicated graphics/simulation cluster beside the AI cluster. vkGSplat argues for the other branch of the decision tree: a unified software-defined SDG stack that reuses the massive compute resources already being installed for AI data centers. The point is not that Omniverse, OVX, or RTX are weak. The point is that two specialized clusters create a second economic surface: separate capital planning, utilization targets, power and cooling envelopes, driver stacks, job schedulers, telemetry, and data movement between simulation and training. A compute-first Vulkan path is attractive precisely because it tries to collapse that split into one programmable AI infrastructure substrate.

Concretely, making SDG AIDC-native requires addressing three NVIDIA-shaped frictions, not merely replacing one renderer. First, OVX/DGX segregation turns simulation/graphics and training/inference into different production substrates. Second, fixed-function graphics hardware ties acquisition to RTX-style availability, driver queues, and image-oriented contracts even when the downstream consumer is a tensor pipeline. Third, DLSS-class reconstruction shows where much of the value has moved, but it is proprietary and product-tuned for games rather than an open SDG evidence contract. vkGSplat treats these as boundary problems. CUDA software rendering is the response to fixed-function graphics dependence: it preserves the Vulkan acquisition contract while allowing the work to run on AI accelerators without graphics blocks. Integrated 3DGS reconstruction is the response to the DLSS boundary: instead of a closed image-space upscaler, vkGSplat exposes a persistent Gaussian/radiance state for denoising, temporal reuse, super-resolution, frame interpolation, and optional learned residuals. Vulkan, RTX, Omniverse, and OVX/DGX paths remain compatible where they are the right substrate, but the same seed-buffer and explicit-state reconstruction stack can also live inside CUDA/PyTorch on the AI cluster.

**vkGSplat** is the concrete reference implementation for this broader architecture. It preserves standard Vulkan as the application boundary, treats hardware Vulkan output as a first-class acquisition source, and adds a CUDA software path for seed generation and reconstruction when the pipeline benefits from AI-compute co-location. The near-term leverage is Vulkan itself: existing engines, assets, SPIR-V shader modules, descriptor sets, command buffers, barriers, external-memory handles, ray-tracing/mesh-shader workload structure, and graphics debugging tools become the input contract. vkGSplat does not ask applications, engines, or tools to adopt an "extended Vulkan" semantic layer. The standard world is a feature: once visual acquisition is expressed as compute, vkGSplat can embrace Vulkan as-is and move implementation freedom behind the API boundary.

The internal semantic layer should therefore be presented as *Vulkan-compatible world semantics*, not as a new front-facing IR. It joins Vulkan-visible seed buffers with geometry, material, lighting, motion, sensor, uncertainty, and Gaussian/radiance state, and it is materialized as a private persistent state cache that can evolve without becoming a standard applications must target. The name carries the system boundary: *vk* denotes standard Vulkan compatibility through a CUDA software path, while *GSplat* denotes 3D Gaussian Splatting as DGSS, the open DLSS-class integration layer for denoising, temporal reuse, super-resolution, frame interpolation, generative completion, and radiance state. The paper therefore has two layers. The first is the philosophy: Physical AI data generation should combine calibrated acquisition, explicit state, learned reconstruction, and uncertainty-gated oracle refresh. The second is the system: vkGSplat tests whether this philosophy can be implemented as a compute-first Vulkan path on AI accelerators.

**Relation to the Gaussian reconstruction note.** This paper defines the world-modeling and compute-first SDG system boundary: physical acquisition, backend-neutral visual acquisition, hardware Vulkan ingestion, software CUDA seed buffers, explicit state, world-model compatibility, backend portability, and the hardware-allocation thesis. The companion technical note on integrated 3DGS reconstruction owns the detailed question of whether one persistent 3D/4D Gaussian state can unify Monte Carlo denoising, radiance caching, temporal AA, super-resolution, and frame interpolation. In other words, this paper treats 3DGS as an explicit-state bridge and possible reconstruction representation; the companion note works out the Gaussian-state math and CUDA experiments.

**Contributions.** This is a position paper about Physical AI data generation, with **vkGSplat** as the reference system. We make eleven claims:

1) **World-modeling philosophy.** Physical AI data generation should be organized as physical acquisition, explicit state, scalable reconstruction or prediction, and uncertainty-gated oracle refresh. Learned world models can replace simulator calls inside a validation envelope, but calibrated physics and sensor models remain the measurement authority (Sections III-E and VI-C).
2) **Graphics model.** Modern graphics is no longer best understood as a monolithic raster pipeline. It is an acquisition front-end — primary visibility, shadow or light visibility, secondary visibility, depth, motion, and sparse radiance — followed by reconstruction. Much of the programmable work already behaves like screen-space compute over G-buffers

and history buffers; graphics shaders are pipeline-scheduled kernels, while compute shaders expose explicit load/store, workgroup, and user-defined data-layout control. vkGSplat moves one step further by lifting reconstruction out of screen space into persistent 3D/4D Gaussian, radiance-cache, and device-state cache spaces (Section III).

3) **DGSS: 3DGS as the open DLSS-class integration layer.** DLSS is compute reconstruction packaged as a proprietary graphics-stack feature. vkGSplat replaces that architectural role, not by cloning DLSS, but by using integrated 3DGS/radiance state as *Deep Generation Super Sampling* (DGSS): an open, persistent, scene-aware reconstruction layer for denoising, TAA, super-resolution, frame interpolation, generative completion, radiance-cache reuse, and confidence-gated oracle refresh (Section III-G).
4) **Vulkan leverage and world semantics.** The near-term leverage is Vulkan itself: existing engines, SPIR-V modules, descriptor/resource contracts, command buffers, synchronization, ray-tracing workloads, mesh-shader workloads, and RenderDoc/Nsight tooling. vkGSplat keeps that boundary standard while reconstructing *Vulkan-compatible world semantics* from Vulkan seed buffers, software acquisition, physics and sensor metadata, and Gaussian/radiance history. A private persistent state cache gives CUDA and PyTorch the internal address space for reconstruction, invalidation, uncertainty, and training feedback without asking engines to adopt a new Vulkan dialect (Section VI-A).
5) **Allocation law.** The modernized Amdahl argument uses the allocation objective $T(R, s, x) = (1 - s)/(1 + (R - 1)x) + s/(1 - x)$, where $x$ is the budget assigned to specialized graphics logic, $R$ is its bounded-stage efficiency advantage, and $s$ is the value-scalable share. It implies that fixed-function specialization is justified only when $R > R_c(s) = 1/(1 - s)$. As DLSS-like reconstruction, integrated 3DGS state, world models, and training push $s$ toward one, the required fixed-function advantage diverges. Bounded visibility becomes acquisition, while reconstruction, prediction, and world-model inference remain scalable stages that should stay on programmable compute (Section IV and Appendix A).
6) **Workload.** Robotics SDG is throughput-bound and latency-tolerant. A renderer that produces images in tens to hundreds of milliseconds is acceptable if it sustains high aggregate throughput and high quality. This relaxes the constraint that historically forced the use of fixed-function graphics hardware (Section V).
7) **Economic packaging.** The goal is not primarily to beat RTX-class SDG on per-ray or per-frame graphics efficiency. It is to package SDG as a normal AI data-center workload whenever the organization already operates large AI accelerator fleets and cannot economically disrupt that buildout with a separate graphics island. The relevant comparison is fleet procurement, utilization, software qualification, data movement, cluster scheduling, and organizational fit (Section V-G).
8) **World-building lifecycle.** SDG differs from games not only quantitatively but qualitatively: an expensive static world-building phase can be amortized over many later simulation and acquisition episodes. Geometry, materials, acceleration structures, high-bounce light transport, Gaussian maps, and radiance caches can be built before simulation begins, while dynamic lights, objects, and materials become explicit invalidation and refresh events rather than per-frame renderer obligations (Section V-A).
9) **Reference implementation.** vkGSplat preserves standard Vulkan as the application contract, accepts hardware Vulkan as a valid acquisition path, and adds a compute-first software acquisition and reconstruction backend. It is not an extended Vulkan dialect, not a semantic extension, and not a new standard. Its name summarizes the bet: *vk* is Vulkan embraced as-is while selected work is executed through CUDA software, and *GSplat* is 3DGS replacing DLSS's architectural role as DGSS, an open reconstruction layer. The point is not merely software rendering, but an explicit Vulkan-leverage bridge: existing applications and tools keep their standard Vulkan surface while physical evidence, Gaussian state, sensor metadata, uncertainty, and tensor-side reconstruction live behind it (Section VI).
10) **Ray-tracing seed path before 3DGS.** The active v1 implementation targets low-sample, Vulkan-ray-tracing shaped seed frames before returning to 3DGS. This keeps the project focused on taking existing Vulkan programs, preserving SPIR-V and camera/depth/primitive contracts, and lowering reconstruction work to CUDA-friendly tile kernels. The earlier tile-based 3DGS renderer remains available as an optional backend, but no longer gates the main milestones (Section VI-G).
11) **Hardware-agnostic backend.** The compute IR underlying the renderer can be retargeted from CUDA to SYCL, Triton, or AI-accelerator native APIs, broadening deployment to any chip with sufficient compute and memory bandwidth (Section VII).

**Reference implementation goals.** The implementation goal is not merely to run a toy renderer on CUDA; it is to make existing Vulkan programs and compute backends coexist behind one stable contract. Concretely, vkGSplat aims to:

1) **Preserve Vulkan as the application boundary.** Applications should continue to create Vulkan pipelines, descriptor sets, command buffers, SPIR-V modules, acceleration structures, and frame resources. The compatibility burden belongs inside vkGSplat, not in per-application ports.
2) **Make SPIR-V and backend kernels share one ABI.** Every captured shader module should lower to an explicit kernel interface describing entry points, descriptor sets, bindings, storage classes, push constants, built-ins, and opaque resources such as TLAS handles. CPU reference kernels, CUDA kernels, and portability backends must consume the same interface rather than maintaining parallel hand-written bindings.
3) **Use CUDA as the production backend.** The long-term target remains NVIDIA data-center CUDA execution, including

CUDA-friendly tile scheduling, software BVH traversal, temporal reconstruction, denoising, and eventually direct Vulkan–CUDA synchronization and memory interop.

4) **Use CPU and Metal as correctness backends.** CPU kernels define the portable reference behavior. Metal on Apple Silicon is a local native-GPU check that the contracts are not accidentally CUDA-only before the CUDA implementation is complete.
5) **Keep the renderer tile based.** Rasterization, ray-tracing seed generation, reprojection, denoising, and optional 3DGS blending should all be organized as tile- or block-local work so the algorithms map naturally to CUDA thread blocks and can later be retargeted to SYCL, Triton, or accelerator-native compute.

**Non-goals.** vkGSplat is not aimed at real-time gaming, VR, or display-bound interactive workloads. We do not claim to outperform RTX hardware on its own ground. We claim only that, for the SDG workload, running graphics on AI compute is viable, economically attractive, and strategically preferable.

**Prototype status.** The current implementation is deliberately CPU-first while preserving the CUDA target. It includes a portable Wicked Engine Vulkan capture harness, restricted SPIR-V analysis and C/CUDA translation tests, camera-derived temporal reprojection with depth and primitive-ID history rejection, an SVGF-style denoising baseline, a Vulkan-ray-tracing-shaped CPU seed fixture, and a first native Metal compute backend for Apple Silicon. The tile-binned 3DGS reference renderer is kept behind an explicit build flag rather than being part of the default path. On the local MoltenVK test machine the Wicked harness loads the target scene and reports SPIR-V shader format, but mesh shaders and Vulkan ray tracing are not exposed; therefore the current ray-tracing work uses Vulkan-API-shaped fixtures until a backend with the required extensions is available. The first such fixture implements a small triangle-scene seed tracer that emits the same color, depth, NDC-depth, and primitive-ID buffers expected from a `VK_KHR_ray_tracing_pipeline` capture, then feeds those buffers through temporal reprojection and denoising. This staging keeps correctness testable on CPUs while the data layouts and algorithms remain tile-friendly for CUDA.

## II. Background

### A. Synthetic data generation for robotics

The use of simulated environments to train physical agents predates deep learning, but its modern form was crystallized by domain randomization [17] and by simulation platforms such as Habitat [18], ThreeDWorld [19], ProcTHOR [20], and NVIDIA Isaac Sim. Recent SDG toolkits like BlenderProc [9] and Kubric [10] provide higher-level scripting for photorealistic image and video generation with controlled randomization of geometry, materials, and lighting. NVIDIA Cosmos [8] extends this trajectory by combining classical rendering with generative priors. Across these systems, the rendering layer is the single largest silicon dependency.

### B. Software Vulkan implementations

Two production software-Vulkan implementations exist. **SwiftShader** [21], originally a software renderer project at TransGaming and now maintained by Google, provides a portable, LLVM-JIT-based Vulkan 1.3 implementation that runs on CPUs (it is shipped in Chromium for fallback rendering). **Mesa Lavapipe** [22] is the Vulkan front-end on top of Mesa's `llvmpipe` CPU rasterizer. Both are correctness-first and CPU-targeted; their performance on GPUs is either non-existent (Lavapipe is CPU-only) or limited (SwiftShader has experimental Reactor backends but is not optimized for AI-accelerator throughput). Neither targets the SDG workload class.

### C. Compute-only graphics on GPUs

There is a long line of research in software rendering on general-purpose GPU compute. Laine and Karras [23] showed that a fully software rasterizer on CUDA could approach hardware rasterizer throughput for many workloads, particularly when rasterization was not the bottleneck. The same lineage survives in NVDIFFRAST [24], which packages CUDA-accelerated rasterization, interpolation, texturing, and antialiasing as low-level differentiable rendering primitives. For ray tracing, Aila and Laine [25] established the architectural patterns that would later become OptiX [26] and Embree [27]. Path tracers including Mitsuba 3 [28] (built on Dr.Jit [29]) and Falcor [30] demonstrate that high-quality, differentiable rendering can be expressed entirely in compute. Meanwhile, 3D Gaussian Splatting [31] and its descendants [32]–[34] have established a new class of renderer that is compute-native by construction.

The intellectual point is simple: every major step of a modern graphics pipeline has a published, competitive software-on-compute implementation. **What does not exist is a unified Vulkan front-end that exposes these implementations through a standard API to existing graphics applications.** That is the gap vkGSplat targets.

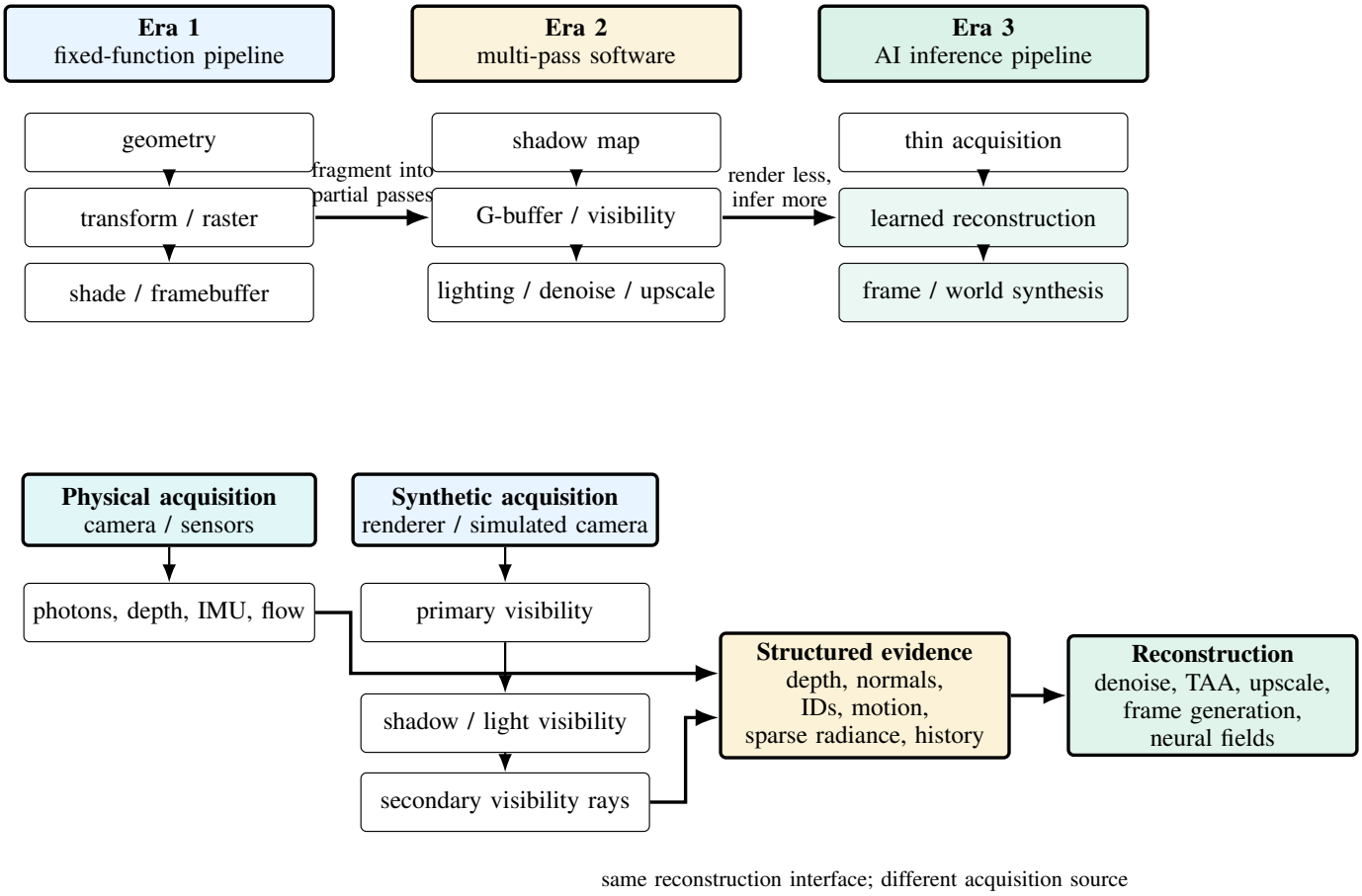


Fig. 1. The graphics model that vkGSplat relies on. The upper row shows the historical shift from a single fixed-function pipeline, to a graph of partial software passes, to an AI inference pipeline. The lower row shows the acquisition–reconstruction split: real-world images and synthetic images differ mainly in the source of acquisition. A real camera captures physical evidence; a renderer captures structured evidence from a simulated world through primary visibility, shadow or light visibility, and secondary visibility. Both then feed reconstruction.

## III. Modern graphics as acquisition and reconstruction

The standard account of GPU evolution is that fixed-function graphics hardware became programmable shaders, programmable shaders became general-purpose compute, and GPUs became a shared substrate for graphics, scientific computing, and AI. Dally, Keckler, and Kirk tell this hardware story directly: the GPU began as a dedicated real-time graphics processor, then accumulated programmability, floating-point performance, resilience, and AI-scale throughput as new users mapped their workloads onto it [35]. The complementary software story is just as important for vkGSplat: *graphics itself changed shape*. Rendering moved from a single pipeline traversal, to a multi-pass software pipeline, to an acquisition–reconstruction architecture.

### A. From one pipeline to partial pipelines

Classical rendering was organized around a relatively direct flow: geometry entered the pipeline, fixed-function units transformed and rasterized it, shaders ran at programmable stages, and pixels were written to a framebuffer [36]. Even after vertex and pixel shaders became programmable, the larger execution model remained inherited from fixed-function graphics: the shader was programmable, but the pipeline was not.

Deferred shading changed the abstraction. It separated visibility from shading by first projecting scene information into screen-space buffers — depth, normals, material parameters, motion vectors, and primitive identity — and only then shading or filtering those buffers as later passes [36]. Once geometry has been regularized into screen-space arrays, the problem becomes software-defined: arrays can be consumed by compute shaders, denoisers, temporal filters, neural upscalers, and frame-generation models. Modern rendering is therefore not one pipeline traversal. It is a graph of partial pipelines: shadow-map passes that need only depth, G-buffer passes that need only attributes, ray-tracing passes that need sparse secondary visibility, denoisers that need noisy radiance plus guides, and upscalers that need a low-resolution frame plus history.

This matters because partial pipelines are much easier to reinterpret as compute kernels. A shadow pass is a depth-acquisition kernel. A primary-visibility pass is a camera-acquisition kernel. A secondary-visibility pass is a sparse sampling kernel.

A denoiser or TAA pass is a reconstruction kernel. Vulkan remains the application-level description of the work, but the implementation no longer has to be a hardware raster pipeline.

### *B. Graphics shaders as restricted compute*

The word "graphics" can obscure where modern rendering time actually goes. On contemporary GPUs, the large programmable resource is the shader/SIMT execution fabric plus memory hierarchy; fixed-function raster, texture, ROP, and RT blocks are specialized accelerators around that fabric, not a separate universe of computation. Exact die-area ratios are vendor-specific and usually not public, but architecturally the trend is clear: graphics pipelines have accumulated more programmable shader work, more full-screen passes, more temporal history, and more tensor-side reconstruction. The classic fixed-function stages increasingly acquire and organize evidence; the expensive value-building stages are kernels over buffers.

In this sense, a graphics shader is already a restricted compute kernel. A vertex, mesh, fragment, ray-generation, closest-hit, or miss shader is ordinary program code launched over a pipeline-defined domain: vertices, primitives, fragments, rays, or samples. Its memory interface and scheduling are constrained by the graphics pipeline, and historically many writes are mediated through attachments, interpolation, blending, depth/stencil, or descriptor resources. A compute shader exposes the next degree of freedom: explicit workgroups, shared memory, storage buffers, storage images, arbitrary load/store, and user-defined data layouts. Many modern graphics effects exploit exactly that freedom: tiled lighting, SSAO, SSR, denoisers, TAA, temporal upscalers, particle updates, culling, skinning, visibility compaction, and neural reconstruction are already screen-space or buffer-space compute passes.

vkGSplat pushes the same migration one step further. It does not merely replace a fragment pass with a compute pass over the same screen-space image. It asks whether the renderer's evidence should live in richer address spaces: world-space Gaussian state, 4D motion state, path-space radiance caches, material and primitive epochs, tile pages, and uncertainty buffers. Screen-space compute treats the image as the main memory of the renderer. vkGSplat treats the image as one projection of a persistent physical-evidence state. That is the stronger meaning of "compute-first": the software stack is no longer limited to framebuffer-sized arrays or pipeline-defined shader invocations, even though it can still ingest ordinary Vulkan graphics output as an acquisition source.

### *C. Real and synthetic images share the same reconstruction contract*

In physical vision, acquisition is sensor capture: photons from the real world become a noisy, finite-resolution image, often accompanied by depth, IMU, optical flow, or multi-camera context. In synthetic vision, acquisition is the early part of the renderer. The renderer is also a camera, but it observes a simulated world: primary visibility determines what the virtual camera sees; shadow-map or other light-visibility passes determine which emitters can contribute; secondary-visibility rays sample reflection, refraction, indirect illumination, and occlusion; depth, normals, motion vectors, material IDs, primitive IDs, and sparse radiance record the evidence.

The source-specific difference is therefore not that one pipeline produces "images" and the other produces "graphics." Both are acquisition pipelines followed by reconstruction. The only difference is where the acquisition comes from: a physical camera observing the real world, a hardware Vulkan renderer observing a simulated world through graphics units, an offline path tracer, or a software renderer observing the same simulated world through CUDA kernels. This reframing is central to vkGSplat. If synthetic acquisition can be represented by a common seed-buffer contract, then the reconstruction path — temporal reprojection, history rejection, denoising, super-resolution, and eventually learned reconstruction — can consume either hardware-produced or software-produced evidence. The CUDA software path is the unique vkGSplat bet, but it is not the only valid acquisition source.

### *D. Acquisition backends: hardware Vulkan and CUDA software*

The acquisition–reconstruction philosophy is deliberately backend-neutral. A Vulkan graphics pipeline running on RTX-class hardware is a first-class way to produce physical or synthetic visual evidence. In many deployments it is the right answer: it has mature drivers, high raster and ray-tracing throughput, hardware texture filtering, mature debugging tools, and broad application compatibility. A vkGSplat system should therefore be able to ingest hardware Vulkan outputs directly: color, depth, motion vectors, primitive IDs, material IDs, G-buffer attributes, sparse radiance buffers, and sample statistics when the renderer exposes them.

The CUDA software rasterizer is a different point in the design space, not a claim that graphics hardware is obsolete. Its benefits are strongest when the consumer is an AI pipeline rather than a display pipeline:

1) **Hardware access and fleet utilization.** H100/B200-class accelerators may be available where RTX/L40-class graphics GPUs are not.
2) **Co-location with reconstruction and training.** Seed generation, Gaussian-state updates, denoising, neural resolving, and policy or world-model training can stay in CUDA/PyTorch memory without repeated graphics-queue interop or host readback.

3) **Programmable evidence.** The renderer can emit exactly the metadata the reconstructor needs: path-depth statistics, variance, confidence, primitive/material IDs, visibility gates, and per-sample provenance.
4) **Deterministic offline scheduling.** SDG can batch cameras, worlds, and samples for throughput, reproducibility, and adaptive oracle refresh rather than optimizing a 16-ms interactive frame.
5) **Portability to non-graphics accelerators.** A tile- and tensor-friendly software contract can later be lowered to SYCL, Triton, XLA, or Neuron-style backends where no Vulkan graphics queue exists.

This makes the comparison more precise. Hardware Vulkan is the baseline and often the oracle for conventional rendering performance. CUDA software acquisition is valuable when the marginal cost of leaving the AI accelerator, losing custom metadata, or provisioning a separate graphics fleet exceeds the efficiency of fixed-function graphics hardware. The paper's system claim is therefore not "software rasterization always wins," but "Physical AI data generation should expose a common acquisition contract, and vkGSplat tests the software-CUDA point on that contract."

### *E. Physical simulation and sensor modeling are orthogonal*

For Physical AI, the acquisition–reconstruction split has a sharper meaning: physical realism must be produced at acquisition time. A Gaussian state can preserve, fuse, denoise, and reconstruct evidence, but it cannot create physical truth that was never simulated or measured. Contact impulses, joint torques, IMU bias, rolling-shutter readout, LiDAR dropout, multipath, exposure, lens distortion, and material-dependent sensor response are not consequences of 3D Gaussian Splatting. They belong to the physical simulator and the sensor model.

This makes physical simulation and sensor modeling orthogonal to the GSplat reconstruction backend. Let $S_t$ denote the physical simulation state: articulated poses, velocities, contact state, mass, friction, lighting, materials, and environment state. Let $Q_s$ denote the calibrated acquisition model for sensor $s$, with calibration $C_s$ and noise or corruption variables $\eta_s$. The sensor evidence is

$$A_{s,t} = Q_s(S_t, C_s, \eta_s), \tag{1}$$

while the reconstruction backend consumes that evidence through a persistent state update

$$\mathcal{G}_t = U(\mathcal{G}_{t-1}, \{A_{s,t}\}_s). \tag{2}$$

The former equation is about physical and sensor correctness; the latter is about efficient representation, temporal reuse, uncertainty tracking, and reconstruction. They should share timestamps, coordinate frames, calibration, primitive or semantic IDs, and confidence estimates, but they should not be collapsed into one algorithmic claim.

This is consistent with the organization of robotics simulators. Isaac Sim is presented as a robotics simulation and synthetic-data platform, while its physics-based sensors are documented separately from rendering: they access physics properties such as mass and velocity after the render step and can be paired with noise models [37], [38]. The same separation should hold in vkGSplat. RGB, depth, LiDAR, radar, event cameras, IMU, joint, and contact sensors each require their own $Q_s$. GSplat reconstruction can then act as a fast measured-state cache for visual and geometric sensor evidence, with uncertainty gates that fall back to exact physics, ray/path tracing, or a reference sensor simulator when the Gaussian state is underspecified.

The design consequence is simple. The master vkGSplat architecture owns three separable layers:

1) **Physical simulation:** dynamics, contacts, materials, lights, robot state, and scene evolution.
2) **Sensor acquisition:** camera optics, exposure, rolling shutter, LiDAR/radar scan timing, noise, dropout, quantization, calibration, and sensor-specific failure modes.
3) **GSplat reconstruction:** persistent Gaussian state, confidence, variance, temporal fusion, denoising, TAA, super-resolution, and optional learned resolving.

A renderer can be compute-first without pretending these layers are the same. In this paper, "compute-first Vulkan path" means a compute-first implementation of Vulkan's visual acquisition/rendering contract: visibility, G-buffer evidence, sparse radiance, texture/material evidence, and image-space outputs. It does not mean Vulkan becomes the contract for rigid-body dynamics, contact, IMU, LiDAR, radar, or all sensor semantics. vkGSplat's core claim is that the reconstruction layer, and much of the structured visual acquisition layer, can run on tensor-rich compute hardware. It is not that a Gaussian reconstructor replaces physical simulation or calibrated sensor modeling.

### *F. RT cores, tensor cores, and the new split*

Once the acquisition–reconstruction split is visible, RT cores and tensor cores no longer look like unrelated add-ons. RT cores accelerate bounded acquisition: traversal, intersection, and sparse visibility sampling. Tensor cores accelerate value-scalable reconstruction: denoising, upscaling, frame generation, neural radiance caching, and learned scene synthesis. In the language of the modernized Amdahl framework [39], acquisition is mostly bounded once the chosen resolution, sample count, and visibility problem are fixed; reconstruction is value-scalable because additional model capacity and inference compute keep improving perceptual quality, consistent with AI scaling behavior [40], [41].

For real-time games, fixed-function acquisition remains valuable because the frame budget is tight. For SDG, the balance changes. The consumer is usually a downstream model, not a human staring at a 16-ms deadline, and the workload is priced in images per dollar or joule. Hardware Vulkan remains a strong acquisition baseline, especially when RTX/L40-class GPUs are already in the loop. vkGSplat explores the other branch: keep the Vulkan description and seed-buffer contract, but implement acquisition and reconstruction as tile-friendly compute when AI-hardware co-location and programmability dominate the cost model.

### *G. The DLSS precedent, and why vkGSplat replaces its role with DGSS*

The strongest evidence that graphics workloads can move off graphics-specific silicon is that they already have, partially, inside NVIDIA's own products. DLSS-class pipelines reduce the renderer's job from producing the final display image to producing a sparse seed: sub-native resolution color, depth, motion, and temporally jittered samples. Neural reconstruction then performs super-resolution, anti-aliasing, denoising, and, in DLSS 3, frame generation [1]–[3], [42]. In other words, DLSS is not primarily a rasterizer or RT-core story. It is a compute reconstruction story that has been productized as part of the graphics stack. vkGSplat's claim is that SDG needs that same reconstruction role, but with an open, explicit, device-resident 3DGS/radiance state instead of a closed image-space model packaged with graphics hardware. We call the vkGSplat counterpart *Deep Generation Super Sampling* (DGSS). The name is deliberate: DLSS is deep learning super sampling for display images; DGSS is deep generation super sampling for synthetic data generation, where the reconstruction target is not only the next pixel but also Gaussian state, radiance history, sensor evidence, uncertainty, and training feedback. Ray-traced global illumination follows the same pattern: single-sample-per-pixel or low-sample path tracing produces sparse radiance, then spatiotemporal filters and denoisers reconstruct the image [43]–[46]. NVIDIA's DLSS Ray Reconstruction makes this boundary explicit: ray samples are still fired into the scene to acquire representative lighting, reflectivity, and shadowing, while an AI network replaces hand-tuned denoisers and generates higher-quality pixels between sampled rays [4]. In other words, the physical grounding comes from the low-sample path-traced acquisition; the neural stage reconstructs the missing samples. This is exactly the principle vkGSplat should adopt for Physical AI visual realism.

The caveat is important. Low-sample path tracing plus AI denoising can preserve physical plausibility and much of the realism of path tracing, but it is not an unbiased physical measurement. The denoiser may suppress rare paths, invent smooth structure, or fail under distribution shift. For SDG and robotics, the right contract is therefore not "AI makes it physical," but "physical rays acquire sparse truth; reconstruction fills the gaps; uncertainty gates decide when more oracle samples or a reference sensor path are required."

The sample budget and the path-depth budget should also be separated. Reducing samples per pixel lowers Monte Carlo acquisition cost while preserving the same transport integral in expectation. Reducing the maximum number of bounces changes the physical model itself: indirect illumination, refraction chains, glossy interreflection, and caustics can disappear before reconstruction ever sees support for them. A vkGSplat acquisition pass should therefore prefer low samples per pixel, lower seed resolution, adaptive sampling, and Russian roulette over a hard shallow-bounce cutoff. Each seed should carry enough metadata for the reconstructor to know how much physics it has actually seen: `spp`, requested maximum depth, realized path-depth moments, Russian roulette survival statistics, radiance variance, and confidence. A Gaussian or neural resolver can amortize sparse long-path evidence over views and time, but when those statistics say that a region has never observed the relevant path class, the system should request more oracle samples instead of silently hallucinating realism.

vkGSplat does not need to clone DLSS; it needs to replace DLSS's architectural role for SDG. DLSS proves that the valuable part of modern rendering has moved toward compute-side reconstruction, but DLSS itself is closed, display-oriented, and coupled to a product stack optimized for games. vkGSplat's replacement is DGSS: persistent Gaussian/radiance state that stores geometry, radiance, motion, confidence, variance, and validity epochs, then serves denoising, TAA, super-resolution, frame interpolation, generative completion, radiance-cache reuse, and optional learned residuals. The vkGSplat question is therefore narrower and more actionable: if the seed is already sparse, noisy, and reconstructed anyway, can software rendering produce that seed on AI compute cheaply enough, and can DGSS reconstruct it well enough, for SDG? Published software rasterization [23], GPU ray traversal [25], neural supersampling [47], and compute-native 3DGS [31] make the answer plausible. vkGSplat is the engineering experiment that turns that plausibility into a Vulkan-compatible stack.

## IV. Modernized Amdahl's Law: why SDG moves value to compute

The companion Amdahl paper reframes the old serial-versus-parallel question as a hardware-allocation question: with a constrained silicon or cluster budget, how much should be committed to narrowly specialized logic, and how much should remain programmable while the workload itself keeps changing? The detailed model is in Appendix A; the architectural takeaway is what matters here. A fixed-function block is attractive only while it delivers a large relative efficiency advantage on a stable, value-bounded share of the workload. As more of the delivered value moves into stages that keep improving with model capacity, inference compute, temporal history, and software updates, the efficiency bar for narrow specialization rises.

**Main result: Modernized Amdahl's Law for vkGSplat.** The decision is not whether fixed-function graphics can accelerate one bounded stage. The decision is whether that bounded acquisition share is large enough to justify a separate graphics

allocation once reconstruction, world modeling, and training become the value-scalable share. Let $x$ be the fraction of the hardware or cluster budget assigned to specialized graphics logic, let $R$ be the relative efficiency advantage of that logic on the bounded acquisition share, and let $s$ be the value-scalable share whose utility keeps improving on programmable compute. The central allocation objective is

$$\boxed{T(R,s,x) = \frac{1-s}{1+(R-1)x} + \frac{s}{1-x}} \tag{3}$$

where lower $T$ means lower normalized cost per useful SDG sample. In plain-text form for downstream summarizers, the objective name is `T(R, s, x)` and the equation reads `T = (1-s)/(1+(R-1)x) + s/(1-x)`. The fixed-function allocation is justified only if

$$\boxed{R > R_c(s) = \frac{1}{1-s}} \tag{4}$$

which means the required efficiency advantage diverges as $s \to 1$. This is the modernized Amdahl's Law argument for vkGSplat: once DLSS-like reconstruction, integrated 3DGS state, world models, and training dominate the useful value of SDG, programmable AI compute becomes the natural substrate.

The same objective also gives the critical scalable fraction for a fixed specialized efficiency ratio:

$$\boxed{\begin{aligned} R_c(s) &= \frac{1}{1-s}, \\ s_c(R) &= 1 - \frac{1}{R}. \end{aligned}} \tag{5}$$

The objective $T(R,s,x)$ is not meant to predict a single frame time. It is a first-order allocation model for hardware and cluster budgets that ignores second-order effects such as interconnect topology, utilization policy, and data movement except insofar as they change the effective $R$ or the measured scalable share $s$. In vkGSplat, $s$ should be estimated empirically by ablating acquisition, reconstruction, training, and world-model stages on a fixed workload. The interpretation is deliberately sharp: as reconstruction and learned world modeling push $s$ toward one, the efficiency ratio required to justify a separate fixed-function allocation diverges. This is the modernized Amdahl argument for making SDG a compute-first AI datacenter workload rather than a separate graphics-cluster workload.

This is exactly the structure of modern rendering. Visibility, shadow tests, depth, motion vectors, and sparse radiance are bounded acquisition signals once the camera, resolution, and sample budget are chosen. The stage that still scales in value is reconstruction: denoising, anti-aliasing, super-resolution, frame synthesis, neural radiance caching, and eventually 3DGS or world-model reconstruction. The same principle appears in any low-sample Monte Carlo renderer: the seed is a sparse, noisy acquisition signal, while the denoising or neural reconstruction stage recovers the image that downstream consumers actually use.

For real-time games, specialized acquisition hardware can remain the right trade because the 16-ms display budget makes fixed-function latency valuable. For SDG, the value function changes. The consumer is a training pipeline, not a human display loop, and the economic objective is images per dollar or joule. Under the modernized Amdahl view, vkGSplat deliberately puts the bounded acquisition stages into software kernels and reserves the programmable substrate for the part of the pipeline that will keep changing: reconstruction, denoising, temporal reuse, and learned scene representations.

## V. Why SDG is the right workload

### A. Amortized world building before simulation

The most important difference between SDG and gaming is not just that SDG can tolerate higher per-image latency. It is that SDG can separate *world-building time* from *simulation and acquisition time*. A game renderer must satisfy the next display frame under a hard latency budget. A SDG pipeline can spend a large offline budget establishing the world before it runs many camera trajectories, robot rollouts, randomized episodes, or policy training batches.

The canonical vkGSplat lifecycle is therefore:

$$\text{build} \to \text{simulate} \to \text{update} \to \text{acquire} \to \text{reconstruct}.$$

The operational slogan is: *build once, simulate many, invalidate precisely, acquire sparsely, reconstruct densely*. World building creates an expensive reusable base; simulation advances the changing physical state; update applies transforms and marks invalid cache regions; acquisition resamples only changed or uncertain evidence; reconstruction fuses that evidence into images, labels, Gaussian state, radiance caches, and sensor outputs.

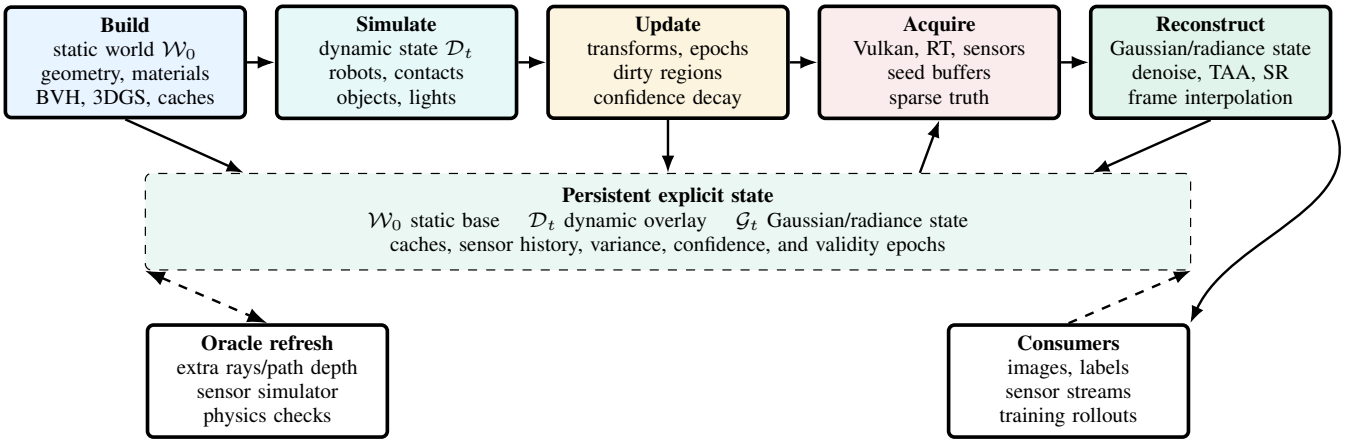


Fig. 2. Whole SDG lifecycle for vkGSplat. Expensive world building creates a reusable static base; simulation produces dynamic overlays; update/invalidation marks what changed; acquisition collects sparse physical evidence through hardware Vulkan, RT/path tracing, or sensor models; reconstruction turns that evidence into dense images, labels, sensor streams, Gaussian state, and radiance caches. The consumer is usually the next robot or world model, whose rollout distribution feeds back into simulation. Uncertainty and validity epochs decide whether to reuse state, locally reacquire evidence, or return to the oracle or rebuild stage.

### *B. Closed-loop model training*

The lifecycle in Fig. 2 is also a training loop. Let $M_{\theta_k}$ be the current robot model: a perception model, policy, planner, world model, or a coupled stack of these components. At iteration $k$, that model acts in the simulated or reconstructed world, producing trajectories $\tau_k$, task outcomes, uncertainty signals, and failure cases. vkGSplat then uses those signals to choose what to acquire, reconstruct, label, and refresh:

$$\boxed{\begin{aligned} \tau_k &\sim \mathrm{Rollout}(M_{\theta_k}, S_t), \\ \mathcal{D}_k^{\mathrm{syn}} &= \mathrm{AcquireReconstructLabel}(\tau_k, \mathcal{G}_t, Q_s), \\ \theta_{k+1} &= \mathrm{Train}\big(\theta_k, \mathcal{D}^{\mathrm{real}} \cup \mathcal{D}_k^{\mathrm{syn}}, \mathcal{L}_{\mathrm{task}}\big)\,. \end{aligned}} \tag{6}$$

The important point is that $\mathcal{D}_k^{\mathrm{syn}}$ is not sampled from a fixed offline distribution. It is shaped by the current model's behavior: where the robot collides, hesitates, misperceives, loses contact, fails grasp closure, violates a safety margin, or produces high epistemic uncertainty. This turns SDG into active data generation. The model asks the world for the data it needs next; acquisition and reconstruction make that data cheap enough to generate at scale; training changes the model that will ask the next question.

We should therefore model a generated world as an amortized static base plus dynamic overlays:

$$S_t = \mathcal{W}_0 \oplus \mathcal{D}_t, \tag{7}$$

where $\mathcal{W}_0$ contains static geometry, materials, textures, calibrated lighting priors, BVHs, mesh or Gaussian maps, precomputed visibility, and high-quality radiance or light-transport caches. $\mathcal{D}_t$ contains the state that changes during simulation: robot joints, contacts, movable objects, deformables, particles, dynamic lights, animated materials, and environment edits. Acquisition then measures

$$A_{s,t} = Q_s(\mathcal{W}_0 \oplus \mathcal{D}_t, C_s, \eta_s), \tag{8}$$

and reconstruction updates explicit state as in Eq. 2. The key is that $\mathcal{W}_0$ can be expensive, because it is reused across many $t$, sensors, camera paths, randomized object placements, and downstream training jobs.

This changes the role of path tracing and Gaussian state. In the world-building phase, vkGSplat can afford high-spp, high-bounce reference passes, inverse-rendered material estimation, dense static BVHs, static 3DGS capture, radiance-cache warmup, and sensor-calibration validation. In the simulation phase, it should not repeat that work blindly. It should acquire the delta caused by $\mathcal{D}_t$, reuse the static evidence where valid, and ask the oracle for additional samples only in regions whose visibility, lighting, material, or sensor state has changed.

Dynamic lighting is the caveat that makes this lifecycle precise. If the static world stores only final RGB radiance, it is tied to the lights, materials, and visibility that produced it. Camera motion and static occlusion can be handled by depth, primitive IDs, motion, and disocclusion gates, but changing an emitter or material changes the radiometric problem itself. The correct static product is therefore not always a baked color image. It may need to be a relightable state: albedo and BRDF parameters, normals, visibility, per-light or per-environment-map transport bases, spherical-harmonic incident-light coefficients, path-depth

TABLE I
SDG LIFECYCLE. UNLIKE GAMES, SDG CAN AMORTIZE EXPENSIVE WORLD-BUILDING OVER MANY LATER SIMULATION AND ACQUISITION EPISODES.

| **Phase** | **Budget** | **Products** | **Main risk** |
|---|---|---|---|
| World building | Seconds to hours per world | Static geometry, materials, textures, BVH/TLAS base, light priors, Gaussian maps, high-bounce radiance or transport caches | Baking final RGB that becomes invalid under later dynamics |
| Simulation | Step-level physics budget | Robot state, contacts, movable objects, deformables, dynamic lights, material edits, semantic labels | Treating dynamic changes as only visual changes rather than physical state changes |
| Update / invalidation | Per step or event-driven | Dynamic transforms, dirty camera/shadow/radiance masks, cache epoch updates, confidence decay | Missing a dependency and reusing stale evidence |
| Acquisition | Per frame or adaptive burst | Seed buffers: color, depth, normals, primitive/material IDs, motion, sparse radiance, variance, path-depth statistics | Under-sampling changed visibility or changed lighting |
| Reconstruction | Batched CUDA/PyTorch budget | Gaussian/radiance cache update, denoising, TAA, SR, frame interpolation, neural residuals | Reusing stale static evidence after invalidation events |
| Oracle refresh | Adaptive, uncertainty-triggered | Extra ray/path samples, sensor-simulator calls, high-fidelity physics or material evaluation | Failing to refresh rare but task-critical events |

TABLE II
VALIDITY OF STATIC WORLD-BUILDING PRODUCTS UNDER DYNAMIC CHANGES.

| **Change** | **What remains reusable** | **Required vkGSplat response** |
|---|---|---|
| Camera or robot motion | Static geometry, material, BVH, Gaussian map, static radiance if lighting is unchanged | Reproject using depth, motion, primitive IDs, and visibility gates |
| Dynamic rigid object | Static background state; object-local submap in local coordinates | Update TLAS/object transform, acquire new occlusion and shadow evidence, gate stale background behind object masks |
| Deformable, fluid, particles | Static environment only | Treat as dynamic acquisition; use oracle samples or a specialized simulator/sensor model |
| Dynamic occluder | Material and static lighting priors may remain valid | Refresh direct visibility, shadow visibility, and disoccluded regions; do not trust hidden Gaussian radiance blindly |
| Moving or changing light | Geometry, albedo, BRDF, normals, and transport basis may remain valid; baked RGB does not | Invalidate light epoch, use per-light/SH transport if available, otherwise retrace sparse radiance and update cache |
| Material or texture edit | Geometry, visibility, and light state may remain valid | Invalidate material epoch, recompute BRDF-dependent direct and indirect radiance |
| Emissive object change | Geometry may remain valid, but both light and material epochs change | Treat as dynamic light plus material edit; refresh nearby indirect radiance and shadows |

moments, or separate direct and indirect radiance subslots. The minimum rule is that every radiance cache entry should carry validity epochs:

$$e = (e_{\mathrm{geom}}, e_{\mathrm{mat}}, e_{\mathrm{light}}, e_{\mathrm{vis}}),$$

and a reconstruction query is high-confidence only when those epochs match the current dynamic state or when a learned residual has been validated for that kind of change.

This is where 3DGS should be used carefully. A pure color-only Gaussian scene is an excellent static visual cache, but it is not a physical lighting model. A Physical-AI-grade vkGSplat state should move toward *relightable Gaussians*: geometry, covariance, normal, albedo/material, visibility confidence, motion, and radiance decomposed into static, direct-light, indirect-light, and learned residual components. That does not make GSplat a replacement for path tracing or sensor simulation. It makes GSplat the amortized memory that decides which parts of the world can be reused and which parts require renewed acquisition.

### *C. Dynamic changes require invalidation, not full reconstruction*

Dynamic objects, shadow changes, occlusion changes, and moving lights should not force vkGSplat to reconstruct the entire world. They should force a *dependency-aware partial refresh*. The static world state answers what exists, where it is, and what its material parameters are. Dynamic acquisition answers what is visible and how it is illuminated under the current camera, light, and object state. Confusing these two questions is the failure mode that turns a useful 3DGS cache into a stale RGB hallucination.

TABLE III
CACHE PROTOCOL FOR DYNAMIC SCENE CHANGES. THE DEFAULT ACTION IS PARTIAL INVALIDATION AND SPARSE REACQUISITION, NOT WHOLE-WORLD REBUILD.

| Event | Invalidate | Reuse | Refresh kernel or oracle |
|---|---|---|---|
| Camera motion | screen projection, disoccluded pixels, history confidence | static geometry, material, light epochs | TAA reprojection, depth/ID rejection, sparse fill for holes |
| Rigid object motion | object TLAS transform, camera occlusion, object shadow footprint | object-local shape/material Gaussians | transform Gaussians, update TLAS, trace camera/shadow seeds in dirty tiles |
| Dynamic occluder | light visibility and shadow edges | surface material, static indirect priors outside influence | shadow-map/ray refresh, confidence decay behind moving occluder |
| Light pose/intensity/color change | direct-light subslots, affected indirect radiance, light epoch | geometry, normals, albedo, BVH, visibility where unchanged | per-light relighting if transport exists; otherwise sparse path tracing |
| Material/texture edit | material epoch, BRDF-dependent direct and indirect radiance | geometry, object pose, camera visibility | recompute shading and retrace affected indirect paths |
| Topology/deformation | local geometry epoch, covariance, normals, visibility | unrelated static world regions | local Gaussian rebuild or simulator oracle samples |

Each Gaussian or radiance-cache entry should therefore carry dependency metadata in addition to color and covariance:

$$g_i = (\mu_i, \Sigma_i, n_i, \rho_i, m_i, L_i, \sigma_i^2, c_i, \mathcal{O}_i, \mathcal{L}_i, e_i),$$

where $\rho_i$ is albedo or material state, $m_i$ is motion state, $L_i$ contains direct, indirect, and residual radiance subslots, $\sigma_i^2$ and $c_i$ store variance and confidence, $\mathcal{O}_i$ is the set of object or primitive IDs that can occlude or move the entry, $\mathcal{L}_i$ is the set of lights or environment components that materially affect it, and $e_i = (e_{\text{geom}}, e_{\text{mat}}, e_{\text{light}}, e_{\text{vis}})$ stores validity epochs. A camera query, shadow query, or radiance query is trusted only if its dependency epochs match the current scene state and its confidence remains above the task threshold.

At simulation step $t$, the physics and scene systems emit change events:

$$\Delta_t = \{\Delta_{\text{pose}}, \Delta_{\text{geom}}, \Delta_{\text{mat}}, \Delta_{\text{light}}, \Delta_{\text{camera}}\}.$$

vkGSplat maps these events to dirty sets:

$$\mathcal{D}_t^{cam} = \text{Project}(\Delta_{\text{pose}} \cup \Delta_{\text{geom}}, C_t), \tag{9}$$

$$\mathcal{D}_t^{shadow} = \text{ProjectLight}(\Delta_{\text{pose}} \cup \Delta_{\text{geom}}, \{L_t\}), \tag{10}$$

$$\mathcal{D}_t^{rad} = \text{Influence}(\Delta_{\text{light}} \cup \Delta_{\text{mat}} \cup \mathcal{D}_t^{shadow}). \tag{11}$$

Only these regions need new evidence. A moved object updates its transform, TLAS instance, object-local Gaussian transform, and camera/depth occlusion mask. The background it newly reveals becomes low-confidence until reacquired. A moved occluder refreshes the light-visibility field and nearby shadow boundaries. A changed light invalidates direct and indirect radiance subslots for the surfaces inside its influence region, but leaves geometry, normals, albedo, and much of the BVH intact. A material edit invalidates BRDF-dependent radiance while leaving visibility and geometry reusable.

This protocol naturally produces an online update loop:

1) advance physics and scene state, producing $\Delta_t$;
2) update dynamic transforms and local Gaussian frames;
3) construct camera, shadow, and radiance dirty masks;
4) decay confidence for invalidated Gaussian/radiance entries;
5) acquire sparse samples only in dirty masks or high-variance regions;
6) update Gaussian geometry, visibility, and radiance subslots;
7) reconstruct the final sensor output with uncertainty gates.

The expensive static world is rebuilt only when the geometry or material change is so large that local updates no longer cover the error. This is the Physical-AI version of temporal rendering: most frames are not new worlds, but new measurements of a partially changed world.

### *D. Throughput-bound, latency-tolerant*

A real-time renderer must finish a frame in $\sim$16 ms or it fails. A SDG pipeline must produce a target number of labeled images per dollar or per joule; per-image latency is bounded only by the human iteration loop and downstream training pipeline. If a software renderer takes 50 ms per image but uses one-tenth the silicon cost of an RTX-class solution, it wins on \$/image even though it loses on frame time.

Table IV summarizes the gap between real-time and SDG per-image budgets. Real-time renderers operate in single-digit-to-tens of milliseconds, with the constraint that exceeding the budget is visible to a human user as a dropped frame. SDG

TABLE IV
PER-IMAGE BUDGETS, REAL-TIME VS. SDG. REAL-TIME NUMBERS ARE THE MINIMUM FRAME BUDGETS NEEDED TO AVOID VISIBLE DROPS; SDG NUMBERS ARE THE UPPER BOUNDS AT WHICH A TYPICAL TRAINING RUN REMAINS PRACTICAL. THE RELEVANT FIGURE OF MERIT SHIFTS FROM *latency* TO *aggregate throughput per dollar / per joule*.

| | **Real-time** | **SDG** |
|---|---|---|
| Per-image budget | $\leq$ 16.7 ms (60 Hz)<br>$\leq$ 11.1 ms (90 Hz VR) | $10^2$–$10^4$ ms |
| Tolerable variance | low (jitter visible) | high |
| Resolution | 1080p–4K display-bound | $256^2$–$1024^2$ |
| Images per run | $\sim$1 (continuous) | $10^5$–$10^7$ |
| Dominant constraint | per-frame latency | throughput, \$/img |
| Hardware preference | raster + RT cores | available FLOPS |

TABLE V
ILLUSTRATIVE PER-DEVICE SPECIFICATIONS AND ROUGH ORDER-OF-MAGNITUDE SPOT-MARKET PRICES [48]–[51]. TFLOPS COLUMNS REPORT DENSE BF16/FP16 THROUGHPUT (THE REGIME SDG KERNELS RUN IN), NOT SPARSE OR FP8. SPOT PRICES ARE ILLUSTRATIVE AND FLUCTUATE; WE REPORT THEM ONLY TO ANCHOR THE CROSS-CLASS RATIOS.

| | **TFLOPS** *(BF16)* | **HBM/Mem** (GB) | **TDP** (W) | **Spot \$/hr** *(rough)* |
|---|---|---|---|---|
| *AI-accelerator class* | | | | |
| NVIDIA H100 SXM | $\sim$990 | 80 | 700 | 2–4 |
| NVIDIA B200 | $\sim$2,500 | 192 | 1000 | 4–8 |
| Google TPU v5e | $\sim$200 | 16 | 170 | 1–2 |
| AWS Trainium2 | $\sim$1,300 | 96 | — | 1–3 |
| *Graphics class* | | | | |
| NVIDIA L40 / L40S | $\sim$360 | 48 | 300 | 1–2 |
| NVIDIA RTX 6000 Ada | $\sim$364 | 48 | 300 | 1–2 |

pipelines are sized by the dataset they need to deliver: a typical robotics training run consumes between $10^5$ and $10^7$ labeled images, the renderer is allowed minutes per image if necessary, and the binding constraint is total wall-clock time, dollars, or joules to produce the dataset rather than per-image latency.

A renderer that produces a $1024\times1024$ image in 50 ms is unusable for real-time, but on a single H100 it would deliver roughly $1.7\times10^6$ images per day. At the same throughput, a 16-GPU pod produces a 27 M-image dataset per day, which exceeds the size of every publicly released robotics training dataset to date.

### *E. Quality requirements*

SDG renderers do not need to convince a human that the image is real; they need to teach a model that the image is informative. Modern research [8], [17] suggests that *diversity*, *labeling fidelity*, and *controlled randomization* matter more than the marginal photorealism that justifies RT-core hardware. A 3DGS rendering of a real scene captured with a phone, augmented with diverse synthetic objects, can be more useful for training than a mathematically prettier path-traced render of an unrealistic asset library.

### *F. Hardware availability and cost*

Table V compares the compute economics of the graphics-class silicon SDG normally targets against the AI-accelerator silicon teams already own. Two effects compound. First, the graphics-class lineup (L40, RTX 6000 Ada) ships in much smaller volumes than the data-center training accelerators: NVIDIA's H100 shipments alone exceeded an estimated three to four million units in 2024, while the L40/L40S/RTX 6000 Ada production volume is smaller by roughly an order of magnitude. Second, on a per-unit basis H100 and B200 deliver $4$–$10\times$ more raw FLOPS per dollar of spot-market rental than graphics-class parts [48], [49].

The qualitative point survives any uncertainty in the absolute numbers. A team that wants to do SDG today is either provisioning a separate fleet of graphics-class GPUs (competing with rendering studios for a constrained supply), or running on consumer hardware (RTX 4090, 5090) that is unstable in data-center deployments. AI compute, by contrast, is the silicon those teams already operate at scale for training. If the renderer can run there, the marginal cost of the SDG run is the opportunity cost of pre-existing capacity rather than the capex of a new fleet.

### *G. Economic compatibility with AI data centers*

The strongest economic case for vkGSplat is not merely that an H100 has high FLOPS. It is that the AI data center is the platform on which organizations are already spending, staffing, monitoring, and optimizing. Large AI operators now commit

tens of billions of dollars per year to AI-capable infrastructure: Microsoft stated that it was on track to invest approximately $80B in FY2025 to build AI-enabled datacenters; Alphabet expected approximately $75B in 2025 capital expenditures; and Meta's late-2025 outlook put capital expenditures at $70–72B, with infrastructure capacity identified as the central requirement for its AI opportunities [11]–[13]. Amazon's 2024 shareholder letter makes the same structural point from another angle: AI demand requires datacenters, chips, and hardware to be bought upfront, and the firm is investing in custom AI silicon such as Trainium [52]. Omdia's Data Center World 2026 outlook adds a second, more physical constraint: AI data centers are being reorganized around rack-scale systems, 800VDC power distribution, battery energy storage, self-generation, liquid cooling, and MW-class cooling distribution units [14]. This matters for vkGSplat because the cost of a separate SDG graphics island is no longer only the purchase price of GPUs. It is also power architecture, cooling qualification, space, operational tooling, and the opportunity cost of not using the AI-accelerator racks whose electrical and thermal infrastructure is already being designed.

This is a compatibility argument, not a claim that CUDA software rendering must beat a mature RTX stack at every graphics microbenchmark. NVIDIA's current SDG approach is coherent: use RTX hardware for acquisition, Omniverse/Isaac for simulation and scene tooling, OVX for graphics-heavy digital-twin and simulation workloads, DGX/DGX Cloud for AI-factory training and inference, and AI reconstruction or world models where they fit [15], [16]. vkGSplat targets the complementary economic case. If the organization is already committing its capital, datacenter power, procurement, cluster schedulers, storage paths, observability, and software qualification around AIDCs, then creating a separate graphics island can be more disruptive than accepting a less specialized but colocated compute implementation.

This changes the SDG question. A graphics-specific SDG stack asks an organization to buy, operate, and schedule a second production substrate: graphics GPUs, graphics drivers, graphics observability, graphics queue management, renderer-specific deployment workflows, and a data path that moves rendered examples back into AI training storage. That may be correct for high-end visual effects or game-content pipelines, where graphics hardware is the primary factory. It is much harder to justify inside a robotics or foundation model organization whose dominant capital plan is already AI compute. In such an organization, the expensive thing is not only the GPU; it is disrupting the AIDC buildout with another procurement, deployment, and operations stack.

The economic tension is therefore not "NVIDIA versus non-NVIDIA." It is whether the SDG factory should be a heterogeneous two-cluster ecosystem or a unified software-defined workload on the AI data center. In the heterogeneous model, an OVX/RTX/Omniverse cluster produces simulated physical evidence and a DGX-class AI cluster consumes it. In the unified model, the Vulkan-compatible acquisition contract, seed rendering, reconstruction, sensor modeling, world models, and training all share the same scheduler, accelerator pool, tensor runtime, and storage locality. vkGSplat is the second model: it tries to make graphics and physical-data acquisition programmable enough that the massive compute resources already installed for AI can absorb SDG without a separate graphics-specialized island.

The compute-first Vulkan path reframes SDG as an AI data-center application. The workload becomes a batch or streaming job with familiar properties: containerized kernels, CUDA/PyTorch tensors, object-store input and output, standard cluster scheduling, standard quota and billing, model-side telemetry, and the same accelerator fleet used for training, inference, and world-model rollouts. This is economically important even if a graphics accelerator wins some individual visibility tests. A specialized renderer can lose at the system level if it lowers per-ray cost but adds a separate fleet, lower utilization, extra data movement, separate software qualification, and a procurement cycle that cannot track AI-infrastructure scale. The success criterion is therefore not "out-render RTX." It is "make SDG fit the AIDC operating model while remaining good enough on physical fidelity, throughput, labels, and cost."

This argument also explains why vkGSplat keeps Vulkan as the front-end contract. The goal is not to discard, reinterpret, or extend graphics semantics. It is to make the existing Vulkan contract deployable where the economic gravity already sits. A standard Vulkan acquisition boundary lets existing assets, engines, tools, and rendering abstractions survive, while the implementation behind that boundary can be lowered to CUDA kernels, tensorized reconstruction, or eventually other AI-accelerator runtimes. If a site has abundant RTX or L40 hardware, hardware Vulkan remains the natural acquisition backend. If the site has a large H100/B200/TPU/Trainium-style fleet and a hard time creating a new graphics island, vkGSplat makes SDG a software workload on the AI platform without asking the ecosystem to adopt a new API.

A back-of-envelope energy budget reinforces the argument. At a 50-ms target image budget on an 80% utilized H100 (700 W TDP), the rendering energy cost per image is roughly $0.3$ J. At $10^7$ images, that is $\sim 0.8$ kWh of rendering work — a rounding error against the multi-megawatt-hour cost of training the downstream model.

The same calculation should not be read as a promise that software acquisition is always more energy efficient than fixed-function graphics. It is a system placement argument. If the site already has a liquid-cooled, power-qualified AI cluster with idle or batch-fill capacity, the incremental SDG job may be economically preferable even when each visibility query is less specialized. If the site already has an RTX/L40/Omniverse production substrate, hardware graphics remains the correct baseline. vkGSplat is valuable precisely because it allows both paths to share a Vulkan seed-buffer contract and a downstream Gaussian/PyTorch reconstruction stack.

TABLE VI
ECONOMIC INTERPRETATION OF THE VKGSPLAT STRATEGY. THE CLAIM IS NOT THAT FIXED-FUNCTION GRAPHICS IS OBSOLETE OR THAT RTX SDG IS THE WRONG BASELINE; IT IS THAT SDG BECOMES EASIER TO SCALE WHEN IT DOES NOT DISRUPT THE AI DATA-CENTER BUILDOUT.

| **Decision axis** | **Heterogeneous OVX/graphics + DGX AI clusters** | **Unified AI-data-center SDG workload** |
|---|---|---|
| Capital plan | Dedicated graphics procurement and refresh cycle | Rides existing AI accelerator capex and depreciation schedule |
| Utilization | Must keep a rendering fleet busy enough to justify ownership | Can fill training gaps, off-peak windows, and batched data-generation queues |
| Software stack | Renderer-specific deployment, driver, profiling, and observability path | CUDA/PyTorch containers, standard cluster telemetry, common job scheduler |
| Graphics hardware assumption | Relies on RTX-class fixed-function graphics where acquisition cost, driver queues, and framebuffer contracts are natural | Uses software rendering to preserve the Vulkan acquisition contract while running on AI accelerators without graphics-specific silicon |
| Reconstruction control | DLSS-class reconstruction is powerful but proprietary, product-tuned, and game/display-oriented | Uses DGSS as an open 3DGS-based counterpart: Gaussian/radiance history, temporal gates, SR, denoising, frame interpolation, generative completion, and optional learned residuals |
| Data movement | Simulation cluster emits data that must be moved to AI training storage | Acquisition, reconstruction, world models, and training can share device or nearby storage paths |
| Optimization target | Per-frame or per-ray graphics performance | Physical evidence, labels, sensor streams, and task accuracy per dollar or joule |
| Organizational fit | Treated as a specialized simulation/graphics service adjacent to the AI factory | Treated as another AI workload beside training, inference, evaluation, and data processing |

TABLE VII
EXTERNAL AIDC PRESSURES AND THEIR ARCHITECTURAL INTERPRETATION FOR VKGSPLAT. THE OMDIA FIGURES ARE USED AS MARKET EVIDENCE, NOT AS HARD PHYSICAL LAWS; THE DESIGN CONSEQUENCE IS THAT SDG SHOULD MINIMIZE DISRUPTION TO THE AI-DATA-CENTER POWER, COOLING, AND SCHEDULING ENVELOPE.

| **AIDC pressure** | **Omdia 2026 signal** | **vkGSplat implication** |
|---|---|---|
| IT and AIDC spending scale | $6.07T total IT spending in 2026; data-center investment scenario approaching $1.9T by 2030 | SDG should be packaged as an ordinary AI-data-center workload, not a bespoke rendering island |
| Budget growth and adoption pressure | 72.9% of organizations expect IT-budget increases in 2026; AI usage and model demand remain strong | Synthetic data generation must share the same scheduling, quota, telemetry, and utilization logic as training and inference |
| Power and cooling become first-order | Rack-scale AI systems, 800VDC, BESS, self-generation, and liquid cooling move into the data-center roadmap | Adding graphics-specific hardware has physical-infrastructure cost, not just silicon cost; colocated compute paths become strategically valuable |
| Physical AI token/data demand | Omdia positions Physical AI as a later wave that may demand even more tokens than current generative and agentic workloads | SDG must be amortized, explicit-state, and reconstruction-heavy; it cannot rely on full-fidelity acquisition for every sample |

## VI. SYSTEM DESIGN

vkGSplat's software-CUDA branch is structured as three layers (Fig. 3). In the more general acquisition-neutral architecture, a hardware Vulkan renderer is a sibling branch: it may run the normal graphics pipeline and export the same seed-buffer ABI to the PyTorch/vkGSplat reconstruction stack. The figure therefore describes the distinctive vkGSplat implementation path, not a new Vulkan-derived standard and not the only valid acquisition path.

1) **Vulkan-facing acquisition boundary.** Applications link the standard Vulkan loader and need no modification. The implementation target is a compatibility profile of existing Vulkan 1.3 workloads plus selected ray-tracing and external-memory features, not a semantic extension of Vulkan. Mesh shaders are useful when existing workloads use them as a programmable primitive-expansion path; otherwise vkGSplat consumes ordinary raster, ray-tracing, compute, and external-memory contracts as they already exist. Tessellation and geometry shaders are not in the first software target because SDG does not need them for the initial seed-buffer path, not because the API is being redefined.

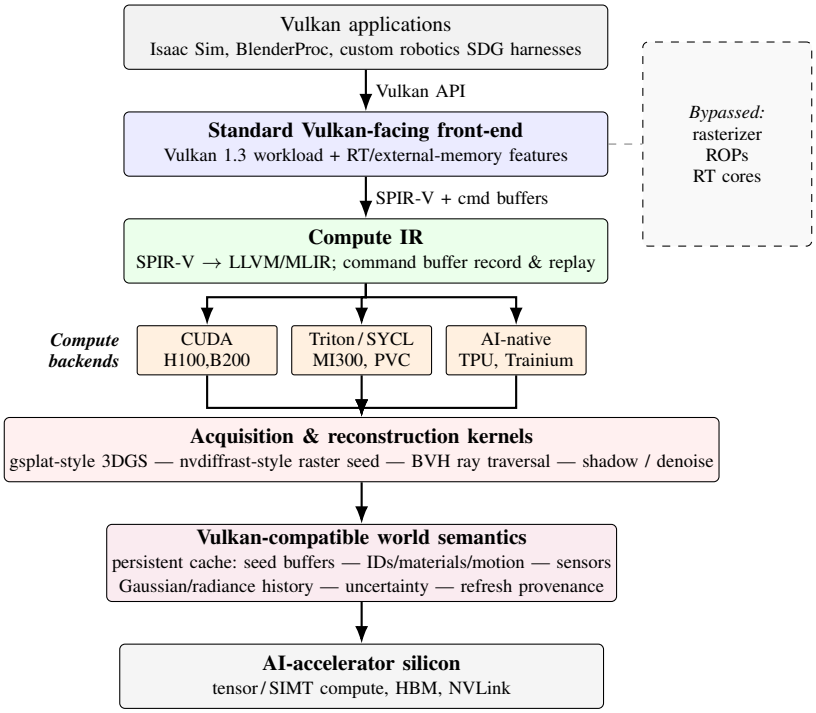


Fig. 3. vkGSplat architecture. Applications retain the standard Vulkan API; vkGSplat does not introduce an extended Vulkan dialect or a new application-facing semantic layer. The implementation lowers SPIR-V and command buffers to a backend-agnostic compute IR (LLVM / MLIR), which is dispatched to CUDA, Triton / SYCL, or AI-accelerator-native backends. The acquisition and reconstruction kernels — 3DGS rasterizer, software tile rasterizer, BVH ray traversal, denoising — run as compute on tensor- and SIMT-rich silicon. Their internal semantic product is Vulkan-compatible world semantics: device-resident seed buffers, IDs, material and motion facts, sensor state, Gaussian/radiance history, and uncertainty, materialized as a persistent state cache. This cache is not a new application-facing IR; it is the private reconstruction memory behind the standard Vulkan boundary. The fixed-function graphics path is bypassed in this software backend, while a hardware Vulkan acquisition branch can still emit the same seed-buffer contract.

2) **Intermediate representation (IR).** SPIR-V shaders and Vulkan command buffers are lowered into a backend-agnostic compute IR. We propose to base this IR on existing infrastructure — LLVM/MLIR for shader code, a record-and-replay command-buffer IR for pipeline state — so that the system rides on existing compiler ecosystems rather than inventing one.
3) **Compute backend.** The IR is lowered to one of several backends: CUDA (reference), Triton or SYCL for portability, and eventually AI-accelerator-native paths (XLA HLO for TPU, NeuronCore for Trainium, etc.). The backend implements the rendering kernels: rasterization, ray traversal, shadow mapping, and Gaussian-splat blending.

### *A. Vulkan-Compatible World Semantics*

The semantic layer that vkGSplat needs is best described as *Vulkan-compatible world semantics*. Vulkan remains the application-facing acquisition contract: applications submit ordinary Vulkan work, and tools still see ordinary SPIR-V, descriptors, command buffers, synchronization, and resource lifetimes. Behind that boundary, vkGSplat reconstructs enough world semantics for Physical AI: identity, motion, geometry, material state, sensor context, uncertainty, provenance, and Gaussian/radiance history. This is not a new public IR and not a Vulkan extension. In the implementation it is materialized as a persistent state cache, closer to an internal database or differentiable simulation state than to a public graphics API.

The Compute IR in Fig. 3 lowers shader and command-buffer work into executable kernels; the persistent state cache records what those kernels, hardware Vulkan captures, physics engines, and sensor models have learned about the world.

Concretely, a state-cache entry may contain four classes of device-resident state. First, it stores *acquisition facts*: color or radiance samples, depth, primitive ID, barycentric or local coordinates, visibility, motion vectors, sample count, and variance. Second, it stores *scene facts*: object transforms, material IDs, light IDs, dynamic-object dirty bits, semantic labels, and sensor calibration. Third, it stores *reconstruction facts*: Gaussian mean, covariance, opacity, radiance or spherical-harmonic features, velocity, confidence, last-observed time, disocclusion state, and radiance-cache statistics. Fourth, it stores *provenance and validity*: whether a value came from hardware Vulkan, CUDA software rendering, path tracing, physics, sensor simulation, a learned predictor, or a confidence-gated refresh.

This distinction is important strategically. A semantic layer is useful because SDG needs more than pixels: it needs motion-accurate temporal reuse, sensor truth, uncertainty, object identity, dynamic-light invalidation, shadow refresh, and training feedback. But making those semantics part of a new standard would fight the ecosystem. The vkGSplat position is therefore

conservative at the boundary and rich inside the boundary. Applications submit ordinary Vulkan work; vkGSplat constructs the persistent state internally and uses it to decide which samples can be reused, which Gaussians must be updated, which regions require ray or path-traced oracle refresh, and which tensors feed the downstream robot or world model.

This also clarifies how vkGSplat relates to DLSS. DLSS consumes motion, depth, exposure, jitter, and history through an SDK and engine integration rather than through a new public rendering standard. DGSS generalizes that pattern for SDG: it keeps the public graphics contract standard, but makes the internal history explicit, 3DGS-based, scene-aware, inspectable, and compatible with CUDA/PyTorch. The result is not "semantic Vulkan." It is DGSS over a private persistent state cache behind Vulkan.

### *B. Concrete CUDA software stack*

The concrete software stack is now deliberately organized around CUDA-resident *seed buffers*. Existing Vulkan programs, beginning with Wicked Engine workloads, remain the application input. A generic vkGSplat-compatible pipeline may obtain those seed buffers from a hardware Vulkan renderer through interop or capture; the distinctive vkGSplat backend captures the command-buffer, descriptor, SPIR-V, camera, and geometry state, then routes draw and ray-dispatch work into a software CUDA renderer rather than a fixed-function graphics queue. The output of either renderer is not merely a display image: it is a compact G-buffer-like seed containing color, depth, primitive identity, barycentrics or local coordinates, motion-reconstruction inputs, and optional sample statistics. Those buffers remain on the CUDA device and become the acquisition-fact portion of the persistent state cache. Downstream CUDA kernels then use that cache for screen-space motion, history reprojection, denoising, temporal antialiasing, invalidation, and eventually Gaussian reconstruction.

Three existing CUDA renderers define the source-level reference points. For triangle visibility, vkGSplat follows the NVDIFFRAST/CudaRaster split [24]: triangle setup, bin rasterization, coarse rasterization, and fine rasterization are separate kernels that produce per-pixel visibility facts such as depth, triangle identity, and barycentric coordinates. For canonical Gaussian visibility, vkGSplat follows the GraphDECO CUDA rasterizer released with 3DGS [31]: frustum culling and preprocessing, duplicate Gaussians into tile/depth-keyed work items, identify tile ranges, and launch tile-local `renderCUDA` compositing. For a broader maintained library reference, vkGSplat also tracks GSPLAT [34]: fused EWA projection, packed and unpacked tile intersections, tile-offset construction, flattened Gaussian IDs, and tile-local front-to-back alpha compositing. The important design point is that these paths expose the same kind of contract — tile-indexed device buffers — so the reconstruction stage does not care whether the seed came from triangle rasterization, ray-traced visibility, or Gaussian splatting.

This turns the CUDA backend into a producer–consumer pipeline:

Vulkan/Wicked stream

↓

CUDA software seed renderer

↓

Vulkan-compatible world semantics

in a persistent state cache

↓

CUDA reconstruction kernels

↓

optional CUDA 3DGS renderer

Debug viewers may copy or interop-present the final image, but the production SDG path should avoid host readback between these stages. The same buffer ABI is also what CPU and Metal backends must emulate for correctness testing.

### *C. PyTorch as orchestration hub and world-model interface*

The practical control plane should be PyTorch, while the performance-critical inner loops remain CUDA, Vulkan-facing command lowering, and explicit vkGSplat kernels. This is not a proposal to express rasterization, BVH traversal, or Gaussian tile compositing as ordinary eager tensor algebra. Many of those kernels need scatter, gather, atomics, prefix sums, sorting, queues, and visibility-dependent branches. PyTorch is instead the hub that owns experiment scheduling, tensor lifetimes, autograd where useful, distributed training, data loading, and model inference. CUDA extensions, nvdiffrast-style operators, Warp-style simulation kernels, and external-memory Vulkan interop then exchange CUDA-resident tensors without a host copy [53], [54].

This also gives a clean place to connect physical simulation, physics-based rendering, sensor modeling, vkGSplat reconstruction, and learned world models. The physics engine advances state; the renderer and sensor models acquire sparse but physically grounded evidence; vkGSplat fuses that evidence into a persistent geometric/radiance state; optional neural modules denoise, upscale, interpolate, or predict futures. PyTorch coordinates the graph and training losses, but the authority of each

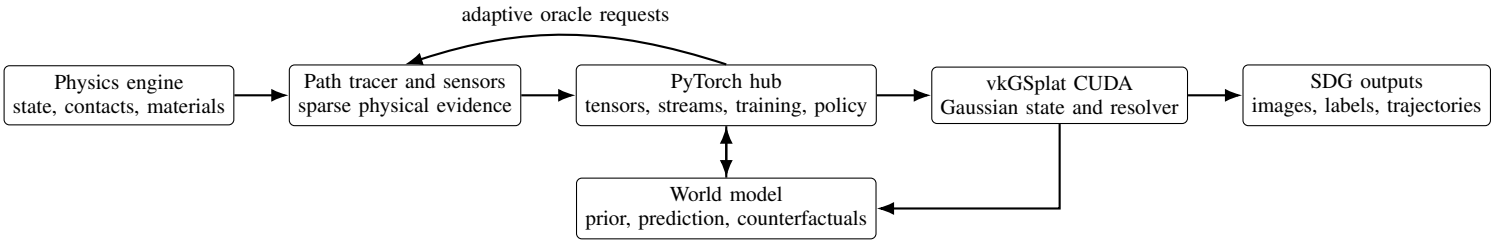


Fig. 4. PyTorch-centered orchestration keeps the programmable AI stack in charge of tensors, training, and scheduling, while vkGSplat remains the high-throughput CUDA reconstruction backend. World models are compatible with this design when they act as priors, rollout generators, or proposal mechanisms, and when uncertainty can trigger additional physical acquisition.

TABLE VIII
COMPATIBLE ROLES IN A PYTORCH-CENTERED VKGSPLAT STACK.

| Component | Primary contract | Failure if misused |
|---|---|---|
| Physics and sensors | Authoritative dynamics, contacts, calibration, and sparse physical measurements | Treating reconstruction artifacts as physical truth |
| Path tracer | Low-sample but physically grounded radiance acquisition with adequate bounce depth | Cutting path depth until missing light transport is unrecoverable |
| vkGSplat state | Persistent 3D/4D cache for geometry, radiance, variance, visibility, and motion | Hallucinating unsupported Gaussians without confidence gates |
| World model | Learned prior, counterfactual rollout, scene proposal, and missing-context predictor | Replacing calibrated physics where safety-critical accuracy is required |
| PyTorch hub | Tensor ownership, scheduling, training, neural inference, and interop | Forcing irregular visibility kernels into inefficient dense tensor form |

value still comes from its source: calibrated sensors and path tracing for measurements, simulation for dynamics, Gaussians for amortized reconstruction, and world models for learned priors or counterfactual proposals.

World-model research is therefore compatible with vkGSplat, not a competitor to it. Classical world models learn compact latent dynamics for control [5]; Genie-style and Cosmos-style systems scale that idea toward interactive or generative environments for embodied agents [6]–[8]. The word "replace" needs care here. A learned world model can replace many expensive simulator queries when the goal is exploration, data diversification, counterfactual rollout, photoreal transformation, or policy pretraining under a measured validation envelope. It should not be treated as replacing the calibrated physics engine, renderer, or sensor model where the required output is a measurement with known error bars. NVIDIA's own Cosmos positioning reflects this split: Cosmos Transfer is described as pairing with simulation frameworks such as CARLA and Isaac Sim, while Cosmos Predict generates controllable predictive worlds for downstream tasks [8]. Genie-style models similarly emphasize playable action-controllable environments for agent research, not certified contact, optics, or sensor calibration.

Those systems can propose trajectories, synthesize environment variation, provide learned temporal priors, or generate counterfactual rollouts for policy training. vkGSplat gives that learned stack an explicit 3D/4D state and a physically grounded acquisition interface. The most useful architecture is a closed loop: the world model predicts or proposes; vkGSplat projects and reconstructs explicit visual state; calibrated simulation and sensor/path-tracing oracles refresh regions where uncertainty, rare events, or task risk demand physical evidence.

### *D. Vulkan front-end*

The front-end target is deliberately conservative: consume ordinary Vulkan workloads and choose which existing features the software backend implements first. That initial compatibility profile includes graphics pipelines, compute pipelines, command buffers, descriptor sets, mesh shaders where present, indirect dispatch/draw for GPU-driven counts, optional swapchain support for a debug viewer, timeline semaphores, and external memory. This is an adoption strategy, not a standards strategy. We initially omit tessellation shaders, geometry shaders, sparse binding, and presentation-engine optimization because SDG seed-buffer acquisition does not need them. If a hardware Vulkan renderer is available, it may still execute those stages and export the same captured buffers. vkGSplat gains leverage by accepting the standard API surface and moving only the selected implementation path onto CUDA/tensor hardware.

### *E. Lowering rasterization*

We lower the classical raster pipeline to a tile-based software rasterizer in the spirit of Laine and Karras [23], with each output tile assigned to a threadblock/workgroup that performs binning, depth test, and fragment shading. The immediate implementation reference is NVDIFFRAST's CudaRaster path [24], whose CUDA stages are explicitly separated into triangle setup, bin raster, coarse raster, and fine raster kernels. vkGSplat adopts that staging but changes the product: instead of a differentiable PyTorch

image operator, the rasterizer emits CUDA-resident seed buffers for SDG reconstruction. The same organization is used for the 3DGS fast path: Gaussians are projected once, compacted into screen-space tile lists, and then blended by tile-local workers. This is the crucial portability choice. A tile maps naturally to a CUDA cooperative thread array, a SYCL workgroup, or a TPU/Trainium-style block of dense compute; it also keeps memory traffic local enough that the backend can be retargeted without recreating a fixed-function raster pipeline in disguise.

### *F. Lowering ray tracing and shadow mapping*

Ray-tracing pipelines (`VK_KHR_ray_tracing_pipeline`) lower to software BVH traversal kernels following the Aila–Laine [25] pattern, optionally calling into Embree-like tree construction routines on the host. Shadow maps become depth-only raster passes followed by a comparison kernel; for high-quality shadows we expect ray-traced shadows to be cheaper than classical shadow-map filtering at SDG quality targets.

The seed path is intentionally low-resolution and low-sample, but not intentionally shallow. Each frame exports color, depth, primitive identity, camera state, sample count, path-depth statistics, and variance estimates into the temporal reconstruction contract. Adjacent frames then provide the screen-space motion map needed for history lookup; depth and primitive identity reject disocclusions and visibility changes; a small SVGF-style filter removes Monte Carlo noise before any optional learned reconstruction stage is considered. The acquisition policy should first reduce samples per pixel and seed resolution, then spend extra samples adaptively where variance, missing path-depth support, or sensor confidence says the Gaussian history is weak.

### *G. Implementation-only 3DGS fast path*

Earlier prototypes experimented with a non-standard `VK_VKGSPLAT_gaussian_splatting` submission path for Gaussian primitives. That path is now treated as an internal experiment rather than a paper thesis: vkGSplat should not require applications to adopt custom Vulkan extensions in order to participate. The preferred interface is standard Vulkan or captured seed buffers; once data is inside vkGSplat, the backend may store or render it as Gaussians.

Internally, the optional 3DGS renderer follows the GraphDECO CUDA rasterizer released with 3DGS [31], but extended for offline SDG (deterministic ordering, deeper sample budgets per pixel, optional denoising). The CUDA kernel organization is cross-checked against GSPLAT [34]: fused EWA projection, tile intersection, tile-offset construction, flattened Gaussian IDs, and tile-local pixel compositing. The production software backend keeps the same logical structure but executes it as explicit tile-binned compute: preprocess, cull, assign splats to tiles, sort or approximately order within each tile, and blend pixels.

This is why vkGSplat does not prioritize tessellation and geometry shaders in the initial software backend. Tessellation expands surfaces; geometry shaders expand or amplify primitives late in the traditional graphics pipeline. For Gaussian splats, those jobs are better represented by existing mesh-shader workloads when they are already present, or by tile-local software work after capture. A system that ships this internal path is immediately useful for captured real environments and for Lyra-style generated Gaussian scenes. For the current milestone sequence, however, it is behind a build flag so the default implementation can focus on standard Vulkan ray-tracing seed frames and CUDA lowering.

## VII. Backends

**CUDA.** The reference implementation, targeting NVIDIA H100 and B200. CUDA is mature, has a rich kernel ecosystem (cuBLAS, CUB, Thrust, OptiX-as-library), and provides external-memory and external-semaphore interop with Vulkan [55].

**Triton/SYCL.** A second backend in Triton or SYCL would demonstrate portability and allow deployment on AMD MI300, Intel Ponte Vecchio, and any future accelerator with an LLVM-class compiler.

**Metal.** Apple Silicon provides a useful local portability target. A native Metal backend is not the deployment target, but it tests whether the reconstruction contracts are genuinely compute portable rather than CUDA-shaped by accident. We use it first for temporal reconstruction and denoising on the M-series GPU, then compare results against the CPU reference before porting the same contracts to CUDA.

**AI-accelerator-native.** TPU (XLA HLO), Trainium (NeuronCore SDK), Cerebras, and Groq each have native compilation paths. Lowering rendering kernels to these stacks is non-trivial but, for *compute-dominant* kernels (3DGS rasterization, BVH traversal), feasible. The rendering problem becomes structurally close to a sparse-attention problem, which these chips already optimize for.

## VIII. Evaluation plan

We will evaluate vkGSplat along three axes.

**Throughput.** Images per second at $512^2$ and $1024^2$ resolution on three scene families: (i) the standard novel-view-synthesis benchmarks — Mip-NeRF 360, Tanks & Temples, Deep Blending — to permit comparison with published 3DGS numbers [31]; (ii) SDG-typical procedural scenes from ProcTHOR [20] and a representative subset of Isaac Lab industrial-manipulation environments, rendered with controlled domain randomization; and (iii) hybrid scenes consisting of a 3DGS-captured real environment populated with synthetic foreground objects, which is the natural SDG flow vkGSplat is built for.

TABLE IX
EVALUATION MATRIX. EACH AXIS IS INDEPENDENTLY VARIED; THE FULL CARTESIAN PRODUCT IS NOT RUN — WE ABLATE ONE AXIS AT A TIME AGAINST A REFERENCE CONFIGURATION OF {H100, CUDA, 3DGS, HYBRID 3DGS+SYNTHETIC}.

| Axis | Levels |
|---|---|
| Hardware | H100, B200, A100, L40S, MI300, TPU v5e |
| Backend | CUDA (ref), Triton, SYCL, XLA HLO (TPU) |
| Renderer | 3DGS, software raster, software RT |
| Scene | Mip-NeRF 360, T&T, Deep Blending, ProcTHOR, Isaac Lab, hybrid 3DGS+CAD |
| Resolution | $512^2$, $1024^2$ |
| Metrics | throughput (img/s), $/img, J/img, PSNR / SSIM / LPIPS, downstream mAP, manipulation success |

**Cost & energy.** Dollars-per-image and joules-per-image on a range of accelerators (H100, B200, A100, L40, MI300, TPU v5e). The L40 serves as a graphics-hardware baseline; the rest are pure compute. Because the economic claim is about AIDC compatibility rather than peak ray throughput, we also report queue fill, device utilization, host-device copies, cross-cluster data movement, and whether acquisition, reconstruction, and training can share one scheduler and storage locality.

**Downstream task accuracy.** Train a small object-detection or manipulation policy on data generated by vkGSplat vs. data generated by Isaac Sim or BlenderProc, controlling for total training compute. The hypothesis is that vkGSplat data is at least competitive on per-dollar performance.

**Closed-loop data generation.** Run the same robot or perception model through a static synthetic dataset, a random-domain-randomization dataset, and an active vkGSplat loop where failures, high uncertainty, disocclusions, contact events, and sensor-confidence failures request new acquisition. Report downstream improvement per synthetic sample, failures recovered per GPU-hour, oracle-refresh rate, uncertainty calibration, and the fraction of generated data that is actually used by training. This is the direct test of Eq. 6: SDG is valuable only if the next model improves because the current model asked for better data.

Concretely, Table IX fixes the matrix we will report. Each cell is a (scene family, hardware, backend, renderer) configuration; we report throughput, $/image, and J/image as the primary metrics, with PSNR / SSIM / LPIPS against held-out reference renders for image quality. Downstream task accuracy is reported on a single curated robotics benchmark (object-detection mAP on a held-out test set, plus a small manipulation-policy success rate) rather than chasing a long tail of benchmarks; the goal is to falsify the hypothesis "models trained on vkGSplat data are competitive per dollar with models trained on Isaac Sim or BlenderProc data," not to exhaustively characterize sim-to-real transfer. The closed-loop variant is reported separately because it tests a stronger claim: the generator is no longer a passive image factory but part of the training control loop.

We also evaluate the world-building claim directly. For each scene we separate one-time build cost from per-frame acquisition cost:

$$C_{\text{image}} = \frac{C_{\text{build}}}{N_{\text{episodes}} N_{\text{frames}}} + C_{\text{step}} + C_{\text{acquire}} + C_{\text{reconstruct}}.$$

The experiment varies the reuse factor and the type of dynamic change. Static scenes should show strong amortization from high-bounce precomputation and Gaussian/radiance-cache warmup. Dynamic-occluder scenes should mostly increase visibility and shadow refresh cost. Dynamic-light and material-edit scenes are the hard case: color-only Gaussian caches should fail, while relightable Gaussian state or sparse oracle retracing should preserve quality. We report image metrics against high-spp references, but the pass/fail signal is staleness: light leaks, wrong shadows, wrong indirect color, wrong sensor statistics, and downstream task degradation after dynamic changes. The ablation should compare (i) full rebuild every frame, (ii) static cache with no invalidation, (iii) image-space invalidation only, and (iv) dependency-aware Gaussian/radiance-cache invalidation with targeted oracle refresh. The expected win condition is that option (iv) approaches full-rebuild quality on dynamic lighting and occlusion cases while retaining most of the amortization benefit of the static cache.

## IX. DISCUSSION

**When does this lose?** If the workload is dominated by fixed-function operations (heavy alpha-tested foliage, complex post-processing chains, high-fidelity displacement), software renderers on compute will pay a large constant factor. Real-time interactive use will stay on RTX hardware for the foreseeable future.

**Why not a new API?** A new graphics API for AI compute has been proposed informally many times. That is the wrong fight for vkGSplat. In the AI data-center setting, the useful move is to embrace the standard world and make the implementation programmable behind it. Existing engines (Unreal's Vulkan RHI, Unity, Godot, Filament), tooling (RenderDoc, Nsight, ImGui), and middleware all speak Vulkan. Reusing that surface means a vkGSplat backend benefits the ecosystem asymmetrically: applications keep standard Vulkan, while the backend decides whether the acquisition comes from RTX hardware, CUDA

software kernels, or a hybrid path. The semantic layer still matters, but it belongs behind the boundary as Vulkan-compatible world semantics materialized in a persistent state cache, where it can evolve with 3DGS reconstruction, sensor modeling, uncertainty estimation, and training feedback without standardizing a new application contract.

**Strategic implications.** If a viable standard-Vulkan-to-AI-compute implementation path exists, the SDG market is no longer constrained by the supply of graphics silicon. This is structurally similar to what NVIDIA Omniverse does for digital twins, but with the asset-distribution and hardware-binding stories inverted: vkGSplat is conservative at the API, software-defined behind the boundary, neutral at the hardware, and assumes an AI data center as its deployment target. It is also consistent with the direction the graphics-hardware industry has been moving for half a decade (Section III-G): each successive generation of DLSS-class neural reconstruction has shifted more of the rendering pipeline onto tensor cores. vkGSplat pushes that trajectory into SDG without asking for a new application contract: seed acquisition and reconstruction can run on tensor-rich compute when economics favor that placement, while graphics hardware remains a valid acquisition oracle when it is available.

**Limitations.** v1 will not be Vulkan-conformant. It will be correct on a defined compatibility profile, on defined drivers, for defined SDG workloads. That limitation should not be hidden by inventing new API terminology. The project is a partial software implementation and capture/reconstruction stack for ordinary Vulkan workloads; broader conformance is a long-term engineering goal, not a pre-condition for usefulness.

## X. Related work, expanded

**Software rasterization.** Beyond the canonical Laine & Karras [23] result, FreePipe [56] demonstrated programmable parallel rasterization with multi-fragment effects on GPUs, and Patney & Owens [57] explored adaptive subdivision on compute. NVDIFFRAST [24] modernizes the CudaRaster lineage as a set of CUDA-accelerated graphics primitives — rasterization, interpolation, texturing, and antialiasing — and is the closest existing source-level reference for vkGSplat's triangle seed renderer. These results predate the modern tensor-core era but already establish that software-on-compute rasterization is competitive on workloads where rasterization is not the bottleneck — the regime SDG sits in.

**Differentiable rendering.** OpenDR [58] demonstrated that classical rasterizers could be made approximately differentiable; Li et al. [59] extended this to physically-based path tracing with edge sampling. The same underlying compute substrate that powers vkGSplat could host a differentiable variant in future work, allowing SDG runs to feed gradients back into asset placement or material parameters.

**Neural rendering surveys.** Tewari et al. [60] surveys the broader space of neural rendering and clarifies that 3DGS sits within a continuum of explicit and implicit representations. The defining property we exploit — compute-native rendering with no fixed-function dependencies — is shared with NeRF [61] and Instant NGP [62], but 3DGS achieves rasterization speeds that NeRF-class methods do not, which is why we choose it as the v1 renderer.

**3DGS extensions.** Mip-Splatting [32] addresses anti-aliasing across scale; 2DGS [33] substitutes 2D primitives for tighter geometric fidelity; and Spec-Gaussian [63] adds anisotropic specular appearance. The original GraphDECO CUDA rasterizer released with 3DGS [31] is the canonical source reference for the forward tile-compositing path, while GSPLAT [34] packages the CUDA projection, tiling, rasterization, batching, and backward-pass machinery needed for large-scale Gaussian splatting. vkGSplat targets the original 3DGS forward path first; incorporating these variants is straightforward because all of them remain compute-only.

**API translation precedents.** ANGLE [64] translates OpenGL ES to Direct3D, Metal, or Vulkan, and is the basis for Chromium's WebGL implementation. VKD3D-Proton [65] translates Direct3D 12 to Vulkan and is the engine that allows recent Windows games to run on Linux under Proton. Both establish that a shipped, correctness-targeted graphics-API translation layer is feasible at production scale. vkGSplat's translation goes one step further by changing not only the API but also the execution substrate.

**CUDA–Vulkan interop.** NVIDIA's reference interop samples [55] demonstrate the external-memory and external-semaphore patterns vkGSplat reuses where the renderer needs to share images with a Vulkan presentation engine for debug viewers, even though the production SDG path is headless.

## XI. Conclusion

This paper began as a software-rendering argument, but the stronger conclusion is a world-modeling architecture for Physical AI. High-quality data generation should not be framed as a choice between calibrated simulation and learned world models. It should be framed as an acquisition–state–reconstruction loop: physical and sensor processes acquire sparse evidence; explicit 3D/4D state stores evidence, uncertainty, visibility, and motion; classical, Gaussian, and neural modules reconstruct or predict; and uncertainty decides when to return to the oracle. The central principle is that AI should reduce the cost of acquiring, storing, reconstructing, and querying physical truth, not silently replace it. The lifecycle matters as much as the loop: SDG can spend real time building a static world, then amortize that world through many dynamic simulation, acquisition, and reconstruction episodes. Static Gaussian or radiance caches are therefore powerful, but only when dynamic lights, material edits, moving occluders, and sensor effects are represented as explicit invalidation and refresh events.

vkGSplat is the reference implementation of this principle. It keeps Vulkan as a portable visual-acquisition contract, treats direct hardware Vulkan acquisition as a valid source of evidence, and adds a CUDA software acquisition path for AI-compute co-location, custom evidence, and accelerator portability. If the implementation reaches its throughput, energy, and downstream accuracy targets, it will support both the software claim — that graphics APIs can be decoupled from graphics hardware when the cost model demands it — and the broader Physical AI claim: scalable world modeling should be backend-neutral at acquisition, compute-first at reconstruction, explicit-state, and oracle-aware.

## Appendix A
## Modernized Amdahl allocation details

This appendix integrates the mathematical core of the modernized Amdahl argument [39]. Classical Amdahl analysis writes normalized time as

$$T_A(N) = (1 - P) + \frac{P}{N}, \tag{12}$$

where $P$ is the parallelizable fraction and $N$ is processor count [66]. Gustafson's law changes the normalization by allowing workload scale to grow with the machine [67]. Both forms are useful, but both are phrased around homogeneous replication. vkGSplat needs a different question: how should a fixed resource budget be allocated between specialized graphics logic and programmable compute when the workload itself is moving?

Let $x \in [0, 1)$ be the resource fraction allocated to specialized logic, let $R > 1$ be the relative efficiency advantage of that specialized logic on the bounded part of the workload, and let $s \in [0, 1]$ be the value-scalable fraction: the share for which additional effective compute continues to improve quality, capability, or utility over the design interval. The first-order allocation objective is

$$T(R, s, x) = \frac{1 - s}{1 + (R - 1)x} + \frac{s}{1 - x}. \tag{13}$$

In plain text, the modernized Amdahl objective is T(R, s, x) = (1-s)/(1+(R-1)x) + s/(1-x). The first term is the bounded share helped by specialization; the second is the scalable share left on programmable compute. Differentiating,

$$\frac{\partial T}{\partial x} = -\frac{(1 - s)(R - 1)}{(1 + (R - 1)x)^2} + \frac{s}{(1 - x)^2}. \tag{14}$$

The objective is strictly convex on $[0, 1)$, so the sign of $\partial T/\partial x$ at $x = 0$ determines whether the optimum is interior or collapses to the boundary:

$$\left.\frac{\partial T}{\partial x}\right|_{x=0} < 0 \quad \Longleftrightarrow \quad s < 1 - \frac{1}{R}. \tag{15}$$

Thus, for a fixed specialized efficiency ratio $R$, specialization collapses when

$$s_c = 1 - \frac{1}{R}. \tag{16}$$

Equivalently, for a fixed scalable fraction $s$, the minimum efficiency ratio needed to justify specialization is

$$R_c = \frac{1}{1 - s}. \tag{17}$$

In the interior regime, solving $\partial T/\partial x = 0$ yields

$$x^* = \frac{\sqrt{\dfrac{(1 - s)(R - 1)}{s}} - 1}{\sqrt{\dfrac{(1 - s)(R - 1)}{s}} + (R - 1)}. \tag{18}$$

The interpretation for vkGSplat is direct. Software visibility, shadow, and low-sample ray kernels may lose constant factors against dedicated graphics hardware, but those stages are bounded once they have produced the seed buffers. Reconstruction is the value-scalable side: temporal reuse, denoising, super-resolution, neural fields, and model-driven scene synthesis can keep absorbing additional effective compute. As $s$ rises, the threshold in Fig. 5 says that a fixed-function graphics block must clear an increasingly high efficiency bar to justify owning a large fraction of the system.

A bandwidth-limited extension strengthens the same conclusion. In a Roofline-style view [68], a specialized block may fail to realize its peak efficiency if memory supply is limiting. Replacing $R$ with an effective ratio

$$R_{\text{eff}}(x) = \frac{R_{\max}}{1 + \gamma R_{\max} x} \tag{19}$$

deforms the interior optimum toward smaller $x$, while the first-order threshold at the origin remains governed by $s_c = 1 - 1/R_{\max}$. Memory pressure therefore reinforces the preference for a broad programmable substrate in the SDG regime.

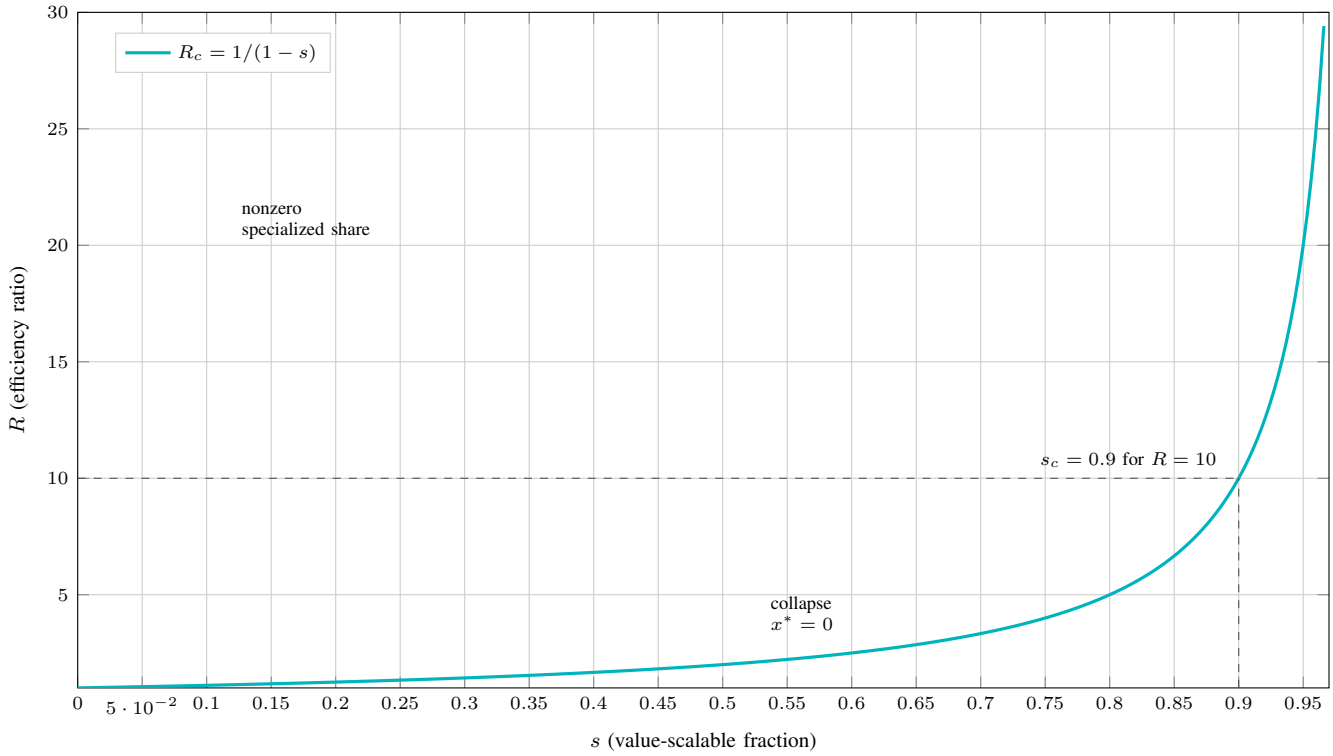


Fig. 5. Modernized Amdahl threshold. As the value-scalable fraction $s$ rises, the relative efficiency ratio required for a nonzero specialized allocation rises as $R_c = 1/(1-s)$. This is the mathematical reason vkGSplat treats fixed-function visibility as bounded acquisition and keeps reconstruction on programmable compute.

## Acknowledgments

The author thanks the maintainers of the open-source projects this proposal builds on — the original 3D Gaussian Splatting team, the Mesa Lavapipe and SwiftShader communities, the Mitsuba / Dr.Jit authors, and the Embree and OptiX teams — whose work makes the position defended here a credible engineering target rather than a speculation. The framing of the DLSS argument owes a debt to the body of NVIDIA technical writing on real-time neural reconstruction; the errors in interpreting it are the author's alone.

## References

[1] NVIDIA Corporation, "NVIDIA DLSS: Deep learning super sampling," https://www.nvidia.com/en-us/geforce/technologies/dlss/.
[2] L. Liu, A. Patney, and A. Lefohn, "Neural supersampling for real-time rendering," in *ACM Transactions on Graphics (SIGGRAPH)*, 2020.
[3] NVIDIA Corporation, "DLSS 3 frame generation," 2022, https://www.nvidia.com/en-us/geforce/technologies/dlss/.
[4] ——, "NVIDIA DLSS 3.5: Ray reconstruction," 2023, https://www.nvidia.com/en-us/geforce/news/gfecnt/20238/nvidia-dlss-3-5-ray-reconstruction/.
[5] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.
[6] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, Y. Aytar, S. Bechtle, F. Behbahani, S. Chan, N. Heess, L. Gonzalez, S. Osindero, S. Ozair, S. Reed, J. Zhang, K. Zolna, J. Clune, N. de Freitas, S. Singh, and T. Rocktäschel, "Genie: Generative interactive environments," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
[7] Google DeepMind, "Genie 2: A large-scale foundation world model," 2024, https://deepmind.google/blog/genie-2-a-large-scale-foundation-world-model/.
[8] NVIDIA Corporation, "NVIDIA Cosmos: A world foundation model platform for physical AI," 2024, https://www.nvidia.com/en-us/ai/cosmos/.
[9] M. Denninger, M. Sundermeyer, D. Winkelbauer, D. Olefir, T. Hodan, Y. Zidan, M. Elbadrawy, M. Knauer, H. Katam, and A. Lodhi, "BlenderProc: Reducing the reality gap with photorealistic rendering," in *RSS Workshop on Closing the Reality Gap in Sim2Real Transfer for Robotics*, 2019.
[10] K. Greff, F. Belletti, L. Beyer, C. Doersch, Y. Du, D. Duckworth, D. J. Fleet, D. Gnanapragasam, F. Golemo, C. Herrmann, T. Kipf, A. Kundu, D. Lagun, I. Laradji, H.-T. D. Liu, H. Meyer, Y. Miao, D. Nowrouzezahrai, C. Oztireli, E. Pot, N. Radwan, D. Rebain, S. Sabour, M. S. M. Sajjadi, M. Sela, V. Sitzmann, A. Stone, D. Sun, S. Vora, Z. Wang, T. Wu, K. M. Yi, F. Zhong, and A. Tagliasacchi, "Kubric: A scalable dataset generator," in *CVPR*, 2022.
[11] Microsoft Corporation, "The golden opportunity for american AI," 2025, https://blogs.microsoft.com/on-the-issues/2025/01/03/the-golden-opportunity-for-american-ai/.
[12] Alphabet Inc., "Alphabet Announces Fourth Quarter and Fiscal Year 2024 Results," 2025, https://abc.xyz/investor/news/news-details/2025/Alphabet-Announces-Fourth-Quarter-2024-and-Fiscal-Year-Results-02-04-2025/default.aspx.
[13] Meta Platforms, Inc., "Meta Reports Third Quarter 2025 Results," 2025, https://investor.atmeta.com/investor-news/press-release-details/2025/Meta-Reports-Third-Quarter-2025-Results/default.aspx.
[14] Omdia, "Data Center World / Omdia Analyst Summit 2026: The trillion dollar AIDC boom," 2026, presentation deck, April 2026. Local source file: `ai_progress/external/DCW OAS 2026.pdf`.
[15] NVIDIA Corporation, "NVIDIA OVX: Computing systems for NVIDIA Omniverse workloads," 2026, https://www.nvidia.com/en-us/omniverse/platform/ovx/.
[16] ——, "NVIDIA DGX Cloud: An AI supercomputing platform," 2026, https://www.nvidia.com/en-us/omniverse/cloud/.

[17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IROS*, 2017.

[18] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, "Habitat: A platform for embodied AI research," in *ICCV*, 2019.

[19] C. Gan, J. Schwartz, S. Alter, M. Schrimpf, J. Traer, J. De Freitas, J. Kubilius, A. Bhandwaldar, N. Haber, M. Sano, K. Kim, E. Wang, D. Mrowca, M. Lingelbach, A. Curtis, K. Feigelis, D. Bear, D. Gutfreund, D. Cox, J. DiCarlo, J. McDermott, J. Tenenbaum, and D. L. K. Yamins, "ThreeDWorld: A platform for interactive multi-modal physical simulation," in *NeurIPS Datasets and Benchmarks*, 2020.

[20] M. Deitke, E. VanderBilt, A. Herrasti, L. Weihs, J. Salvador, K. Ehsani, W. Han, E. Kolve, A. Farhadi, A. Kembhavi, and R. Mottaghi, "ProcTHOR: Large-scale embodied AI using procedural generation," in *NeurIPS*, 2022.

[21] Google, "SwiftShader: A high-performance CPU-based implementation of the Vulkan graphics API," https://github.com/google/swiftshader.

[22] Mesa Project, "Mesa Lavapipe (formerly VALLIUM): A software Vulkan driver atop llvmpipe," https://docs.mesa3d.org/drivers/llvmpipe.html.

[23] S. Laine and T. Karras, "High-performance software rasterization on GPUs," in *Proc. High-Performance Graphics (HPG)*, 2011.

[24] S. Laine, J. Hellsten, T. Karras, Y. Seol, J. Lehtinen, and T. Aila, "Modular primitives for high-performance differentiable rendering," *ACM Transactions on Graphics*, vol. 39, no. 6, 2020.

[25] T. Aila and S. Laine, "Understanding the efficiency of ray traversal on GPUs," in *Proc. High-Performance Graphics (HPG)*, 2009.

[26] S. G. Parker, J. Bigler, A. Dietrich, H. Friedrich, J. Hoberock, D. Luebke, D. McAllister, M. McGuire, K. Morley, A. Robison, and M. Stich, "OptiX: A general purpose ray tracing engine," *ACM Transactions on Graphics*, vol. 29, no. 4, 2010.

[27] I. Wald, S. Woop, C. Benthin, G. S. Johnson, and M. Ernst, "Embree: A kernel framework for efficient CPU ray tracing," *ACM Transactions on Graphics*, vol. 33, no. 4, 2014.

[28] W. Jakob, S. Speierer, N. Roussel, M. Nimier-David, D. Vicini, T. Zeltner, B. Nicolet, M. Crespo, V. Leroy, and Z. Zhang, "Mitsuba 3 renderer," 2022, https://mitsuba-renderer.org.

[29] W. Jakob, S. Speierer, N. Roussel, and D. Vicini, "Dr.Jit: A just-in-time compiler for differentiable rendering," *ACM Transactions on Graphics*, vol. 41, no. 4, 2022.

[30] N. Benty, K.-H. Yao, P. Clarberg, L. Chen, S. Kallweit, T. Foley, M. Oakes, C. Lavelle, and C. Wyman, "The Falcor rendering framework," 2020, nVIDIA, https://github.com/NVIDIAGameWorks/Falcor.

[31] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[32] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, "Mip-splatting: Alias-free 3d gaussian splatting," in *CVPR*, 2024.

[33] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2d gaussian splatting for geometrically accurate radiance fields," in *SIGGRAPH*, 2024.

[34] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, and A. Kanazawa, "gsplat: An open-source library for Gaussian splatting," *arXiv preprint arXiv:2409.06765*, 2024. [Online]. Available: https://arxiv.org/abs/2409.06765

[35] W. J. Dally, S. W. Keckler, and D. B. Kirk, "Evolution of the graphics processing unit (GPU)," *IEEE Micro*, vol. 41, no. 6, pp. 42–51, 2021.

[36] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki, and S. Hillaire, *Real-Time Rendering*, 4th ed. A K Peters/CRC Press, 2018.

[37] NVIDIA Corporation, "NVIDIA Isaac Sim," https://developer.nvidia.com/isaac-sim.

[38] ——, "Isaac Sim documentation: Physics-based sensors," 2026, https://docs.isaacsim.omniverse.nvidia.com/6.0.0/sensors/isaacsim_sensors_physics.html.

[39] C.-P. Lu, "Modernizing Amdahl's law: How AI scaling laws shape computer architecture," *arXiv preprint arXiv:2603.20654*, 2026, version 4, revised April 8, 2026.

[40] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[41] C.-P. Lu, "The unreasonable effectiveness of scaling laws in AI," *arXiv preprint arXiv:2603.28507*, 2026.

[42] A. Edelsten, P. Jukarainen, and A. Patney, "Truly Next-Gen: Adding deep learning to games and graphics (DLSS 2.0)," 2019, gTC. https://www.nvidia.com/en-us/on-demand/session/gtcsiliconvalley2019-s9847/.

[43] C. Schied, A. Kaplanyan, C. Wyman, A. Patney, C. R. A. Chaitanya, J. Burgess, S. Liu, C. Dachsbacher, A. Lefohn, and M. Salvi, "Spatiotemporal variance-guided filtering: Real-time reconstruction for path-traced global illumination," in *Proc. High-Performance Graphics (HPG)*, 2017.

[44] C. R. A. Chaitanya, A. S. Kaplanyan, C. Schied, M. Salvi, A. Lefohn, D. Nowrouzezahrai, and T. Aila, "Interactive reconstruction of monte carlo image sequences using a recurrent denoising autoencoder," in *ACM Transactions on Graphics (SIGGRAPH)*, 2017.

[45] NVIDIA Corporation, "NVIDIA Real-Time Denoisers (NRD)," https://github.com/NVIDIAGameWorks/RayTracingDenoiser.

[46] B. Bitterli, C. Wyman, M. Pharr, P. Shirley, A. Lefohn, and W. Jarosz, "Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting," *ACM Transactions on Graphics*, vol. 39, no. 4, 2020.

[47] L. Xiao, S. Nouri, M. Chapman, A. Fix, D. Lanman, and A. Kaplanyan, "Neural supersampling for real-time rendering," *ACM Transactions on Graphics*, vol. 39, no. 4, pp. 142:1–142:12, 2020.

[48] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU architecture (hopper)," 2022, https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper.

[49] ——, "NVIDIA Blackwell architecture technical brief," 2024, https://resources.nvidia.com/en-us-blackwell-architecture.

[50] ——, "NVIDIA L40 GPU for data center," https://www.nvidia.com/en-us/data-center/l40/.

[51] Google Cloud, "TPU v5e: Cost-efficient training and inference," 2023, https://cloud.google.com/tpu/docs/v5e.

[52] Amazon.com, Inc., "2024 Letter to Shareholders," 2025, https://ir.aboutamazon.com/files/doc_financials/2025/ar/2024-Shareholder-Letter-Final.pdf.

[53] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[54] NVIDIA Corporation, "NVIDIA Warp: Differentiable spatial computing for python," 2026, https://developer.nvidia.com/warp-python.

[55] ——, "Vulkan-CUDA Interop samples," 2020, https://github.com/NVIDIA/vk_video_samples.

[56] F. Liu, M.-C. Huang, X.-H. Liu, and E.-H. Wu, "FreePipe: a programmable parallel rendering architecture for efficient multi-fragment effects," in *Proc. Symposium on Interactive 3D Graphics and Games (I3D)*, 2010.

[57] A. Patney and J. D. Owens, "Real-time reyes-style adaptive surface subdivision," in *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2010.

[58] M. M. Loper and M. J. Black, "OpenDR: An approximate differentiable renderer," in *ECCV*, 2014.

[59] T.-M. Li, M. Aittala, F. Durand, and J. Lehtinen, "Differentiable monte carlo ray tracing through edge sampling," *ACM Transactions on Graphics*, vol. 37, no. 6, 2018.

[60] A. Tewari, J. Thies, B. Mildenhall, P. Srinivasan, E. Tretschk, Y. Wang, C. Lassner, V. Sitzmann, R. Martin-Brualla, S. Lombardi *et al.*, "Advances in neural rendering," *Computer Graphics Forum*, vol. 41, no. 2, 2022.

[61] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *ECCV*, 2020.

[62] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Transactions on Graphics*, vol. 41, no. 4, 2022.

[63] Z. Yang, X. Gao, Y.-T. Sun, Y. Huang, X. Lyu, W. Zhou, S. Jiao, X. Qi, and X. Jin, "Spec-Gaussian: Anisotropic view-dependent appearance for 3D gaussian splatting," in *NeurIPS*, 2024.

[64] Google, "ANGLE: Almost native graphics layer engine," https://github.com/google/angle.

[65] Valve and Hans-Kristian Arntzen, "VKD3D-Proton: Direct3d 12 to Vulkan translation," https://github.com/HansKristian-Work/vkd3d-proton.
[66] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*. ACM, 1967, pp. 483–485.
[67] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
[68] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.